ADAPTIVE POST-QUANTUM CRYPTOGRAPHY ARCHITECTURE FOR CRITICAL INDUSTRIAL IOT

by
Alperen Ali Tunçay

Engineering Project Report

Yeditepe University
Faculty of Engineering and Natural Sciences
Department of Computer Engineering
2025

ADAPTIVE POST-QUANTUM CRYPTOGRAPHY ARCHITECTURE FOR CRITICAL INDUSTRIAL IOT

APPROVED BY:

Asst. Prof. Dr. Onur Demir .....
(Supervisor)

Asst. Prof. Dr. Esin Onbaşıoğlu .....

Asst. Prof. Dr. Tacha Serif .....

DATE OF APPROVAL: 02/01/2026

ACKNOWLEDGEMENTS

First of all I would like to thank my advisor Asst. Prof. Dr. Onur Demir for his guidance and support throughout my project.

Also, I would like to thank my parents for their support and encouragement throughout my education up to the present.

ABSTRACT

ADAPTIVE POST-QUANTUM CRYPTOGRAPHY ARCHITECTURE FOR CRITICAL INDUSTRIAL IOT

The security of Critical Infrastructure (CI) and Industrial Control Systems (ICS) faces an imminent existential threat from the advancement of quantum computing. Long-lived telemetry data generated by sensors in sectors such as nuclear energy and power grids is vulnerable to ”Harvest Now, Decrypt Later” attacks, where adversaries capture encrypted traffic today to decrypt it retrospectively once Cryptographically Relevant Quantum Computers (CRQCs) become available. This project addresses this vulnerability by designing and implementing a hardware-aware, adaptive Post-Quantum Cryptography (PQC) architecture. The system simulates a heterogeneous industrial environment where a resource-constrained ESP32 microcontroller acts as a legacy field sensor, and a high-performance Raspberry Pi 5 serves as a modern edge gateway.

By integrating the NIST-standardized ML-KEM-768 algorithm[1], the study develops two distinct protocols: a ”Quantum One-Time Pad” utilizing segmentation and continuous rekeying for the microcontroller, and a high-throughput ”Stream Cipher” using SHAKE256 for the gateway. Experimental results demonstrate that the ESP32 implementation achieves information-theoretic security for payloads with a stable processing latency of 23.01 ms per chunk, falling within the acceptable tolerance for industrial monitoring. Furthermore, entropy analysis confirms the high quality of the generated key material (7.999 bits/byte), validating the system’s suitability for securing critical telemetry against future quantum threats without requiring a complete hardware overhaul.

ÖZET

KRİTİK ENDÜSTRİYEL IOT İÇİN UYARLANABİLİR POST-KUANTUM KRİPTOGRAFİ MİMARİSİ

Kritik Altyapı ve Endüstriyel Kontrol Sistemlerinin güvenliği, kuantum hesaplamadaki ilerlemelerle birlikte yakın ve varoluşsal bir tehditle karşı karşıyadır. Nükleer enerji ve elektrik şebekeleri gibi sektörlerdeki sensörler tarafından üretilen uzun ömürlü telemetri verileri, saldırganların şifreli trafiği bugün kaydedip gelecekte güçlü kuantum bilgisayarlar erişilebilir olduğunda çözmeyi hedeflediği ”Şimdi Hasat Et, Sonra Çöz” saldırılarına karşı savunmasızdır. Bu proje, donanım farkındalıklı ve uyarlanabilir bir Kuantum Sonrası Kriptografi (PQC) mimarisi tasarlayıp uygulayarak bu güvenlik açığını gidermeyi amaçlamaktadır. Sistem, kaynak kısıtlı bir ESP32 mikrodenetleyicisinin eski nesil bir saha sensörü olarak görev yaptığı ve yüksek performanslı bir Raspberry Pi 5’in modern bir uç ağ geçidi olarak hizmet verdiği heterojen bir endüstriyel ortamı simüle etmektedir.

NIST standardı olan ML-KEM-768 algoritmasını entegre eden bu çalışma, iki farklı protokol geliştirmiştir: Mikrodenetleyici için segmentasyon ve sürekli anahtar yenileme kullanan bir ”Kuantum Tek Kullanımlık Şerit (One-Time Pad)” ve ağ geçidi için SHAKE256 kullanan yüksek verimli bir ”Akış Şifresi”. Deneysel sonuçlar, ESP32 uygulamasının chunk başına 23.01 ms’lik kararlı bir işlem gecikmesi ile endüstriyel izleme toleransları dahilinde kaldığını ve yükler için bilgi kuramsal güvenlik sağladığını göstermektedir. Ayrıca, entropi analizi üretilen anahtar materyalinin yüksek kalitesini (7.999 bit/bayt) doğrulayarak, sistemin mevcut donanımları değiştirmeden kritik telemetri verilerini gelecekteki kuantum tehditlerine karşı koruyabileceğini kanıtlamaktadır.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS iii
ABSTRACT iv
ÖZET v
LIST OF FIGURES ix
LIST OF TABLES x
LIST OF SYMBOLS/ABBREVIATIONS xi
1. INTRODUCTION 1
1.1. Existing PQC for IoT Implementations 2
1.2. Terms 2
1.3. Problem Definition: The Quantum Threat to Critical Infrastructure 6
1.3.1. The ”Harvest Now, Decrypt Later” Attack Vector 6
1.4. Purpose and Scenario Description 7
1.5. Report Organization 8
2. BACKGROUND 9
2.1. Cryptographic Primitives and Hardness Assumptions 9
2.1.1. Classical Public-Key Cryptography (PKC) 9
2.2. Quantum Computing Fundamentals 10
2.2.1. The Qubit and Superposition 10
2.2.2. Superposition and Parallelism 11
2.2.3. Quantum Gates and Entanglement 11
2.3. Cryptographic Hardness Problems 12
2.3.1. The Vulnerability: Integer Factorization and DLP 12
2.3.2. Shor’s Algorithm: The Mathematical Weapon 13
2.4. Post-Quantum Cryptography: Lattice Theory 13
2.4.1. Lattices in Euclidean Space 14
2.4.2. The Learning With Errors (LWE) Problem 14
2.5. Standards and Regulations 15
2.5.1. NIST PQC Standardization Process 15
2.5.2. Industrial Standards: IEC 62443 15
2.6. Hardware Architecture Analysis 16
2.6.1. ESP32-WROOM-32 Internal Architecture 16
2.6.2. Raspberry Pi 5 Architecture 17
3. ANALYSIS AND METHODOLOGY 18
3.1. System Requirements Specification 18
3.1.1. Functional Requirements (FR) 18

3.1.2. Non-Functional Requirements (NFR) 19
3.2. Threat Modeling for Industrial Control Systems 21
3.2.1. Man-in-the-Middle (MitM) Attack Analysis 21
3.2.2. Replay Attack Analysis 22
3.3. Methodology: The ”Quantum One-Time Pad” 23
3.3.1. Theoretical Basis 24
3.3.2. Adaptation for IoT 24
4. DESIGN AND IMPLEMENTATION 25
4.1. Adaptive Software Architecture 25
4.2. Finite State Machine (FSM) Logic 26
4.3. System Interaction and Threat Defense Design 27
4.3.1. Designing Against Replay Attacks 28
4.3.2. Designing Against Man-in-the-Middle (MitM) 29
4.4. Legacy Sensor Implementation (ESP32) 29
4.4.1. Memory Management and Task Isolation 30
4.4.2. The ”Chunking” Algorithm (Quantum One-Time Pad) 30
4.4.3. Protocol Flow and Timing 31
4.5. Modern Gateway Implementation with Raspberry Pi 5 32
4.6. Application Layer Protocol Design 32
4.6.1. Packet Structure Definition 33
4.6.2. Payload Bit-Map 33
4.7. Security Rationale of Packet Fields 34
4.7.1. Mitigation of Traffic Analysis via Padding 34
4.7.2. Time-Window Constraint for Replay Defense 34
4.7.3. Fail-Secure Error Handling Policy 35
5. TEST AND RESULTS 36
5.1. Experimental Testbed Configuration 36
5.1.1. Hardware Specifications 37
5.1.2. Measurement Instrumentation 37
5.2. Validation of the ”Quantum One-Time Pad” 38
5.3. Performance and Latency Analysis 39
5.4. Gateway Performance Under Load 40
5.4.1. Test Scenarios Definition 40
5.5. Security Verification and Attack Simulations 43
5.5.1. Test 1: Replay Attack Mitigation 43
5.5.2. Test 2: Integrity and Tampering Checks 44
6. CONCLUSION 45
6.1. Summary of Achievements 45

6.2. Future Work 46
Bibliography 48

LIST OF FIGURES

Figure 4.1. System Architecture Overview showing Dual-Lane Strategy 26

Figure 4.2. Use Case Diagram: System Actors and Security Defenses 28

Figure 4.3. Sequence Diagram of the PQC Transmission Protocol 31

Figure 5.1. Processing Latency Distribution on ESP32 39

Figure 5.2. Latency Stability across Test Scenarios (Pi 5) 41

Figure 5.3. CPU Utilization Stability across Test Scenarios (Pi 5) 42

LIST OF TABLES

Table 3.1. Non-Functional Requirements for Industrial IoT PQC 20

Table 4.1. LQSP Packet Structure (Total Size: 1120 Bytes) 33

Table 4.2. Decrypted Payload Internal Structure 33

Table 5.1. Entropy Analysis of ESP32 Generated Keys 38

Table 5.2. Performance Comparison: PQC vs. Classical AES (ESP32) 40

Table 5.3. Raspberry Pi 5 Performance Under PQC Load 41

LIST OF SYMBOLS/ABBREVIATIONS

IoT	Internet of Things
IIoT	Industrial Internet of Things
SCADA	Supervisory Control and Data Acquisition
PQC	Post-Quantum Cryptography
KEM	Key Encapsulation Mechanism
NIST	National Institute of Standards and Technology
ML-KEM	Module-Lattice-Based Key Encapsulation
OTP	One-Time Pad
PKI	Public-Key Infrastructure
CRQC	Cryptographically Relevant Quantum Computer
MitM	Man-in-the-Middle Attack
RTOS	Real-Time Operating System
RPi	Raspberry Pi
ESP32	A low-power system on a chip microcontroller with integrated Wi-Fi and dual-mode Bluetooth

1. INTRODUCTION

The 21st century has witnessed an unprecedented convergence of physical and digital systems, fueled largely by the rapid proliferation of the Internet of Things (IoT). What began as a network of simple connected devices has evolved into the Industrial Internet of Things (IIoT), the critical backbone of modern infrastructures ranging from smart power grids and water treatment plants to nuclear power stations and autonomous production lines [2]. These systems, collectively known as Operational Technology (OT), rely on continuous streams of data—telemetry—to make real-time decisions that affect the physical world. However, this extreme connectivity comes with a significant cost: expanded attack surfaces and increased susceptibility to cyber-physical threats. Every industrial process that becomes digitized also becomes a potential cyber-attack vector, making the security of strategic infrastructure a matter of national and global security.

Simultaneously, we stand on the brink of a second revolution: Quantum Computing. While quantum computers promise to solve problems that are impossible for classical machines, they also pose an existential threat to the cryptographic foundations that secure the digital world. The security protocols that protect our bank accounts, military communications, and critical infrastructure today—namely RSA and Elliptic Curve Cryptography (ECC)—rely on mathematical problems that quantum computers could solve with trivial effort. Utilizing the properties of quantum bits (qubits) such as superposition and entanglement, these next-generation processors have the potential to solve complex problems like prime factorization and discrete logarithms, which ensure the security of classical encryption methods, in a matter of seconds.

This chapter introduces the conflict between these two technological developments and discusses how digital infrastructures must be prepared for the post-quantum era. It sets the stage for the project, focusing specifically on the vulnerability of legacy industrial hardware. It outlines the motivation behind targeting the ”Harvest Now, Decrypt Later” (HNDL) threat and details the specific objectives and organization of this report. Furthermore, it provides a comprehensive breakdown of the existing work in the literature and the fundamental terminology that will be used in the following chapters of the report, establishing the academic and technical framework of the proposed system.

1.1. Existing PQC for IoT Implementations

The transition towards quantum-resistant IoT systems has sparked numerous research and development efforts worldwide, focusing on optimizing lattice-based and code-based cryptography for embedded environments[3]. These existing implementations provide a critical baseline for understanding the performance trade-offs required when deploying Post-Quantum Cryptography (PQC) on hardware with limited resources. In the literature, various architectures have been proposed to address the imbalance between the restricted RAM and clock speed capacities of microcontrollers and the large key sizes of PQC algorithms. While most of these studies aim to update standard TLS protocols with PQC algorithms, significant bottlenecks still exist in areas where low latency is critical, such as Industrial IoT.

Frameworks like the *Open Quantum Safe (OQS)* project [4] and *TinyKEM* [5] have demonstrated that lattice-based schemes are feasible on microcontrollers, albeit requiring significant memory optimizations. While *wolfSSL* and *OpenSSL* now enable PQC-based TLS 1.3 handshakes for edge gateways, these implementations remain too resource-intensive for simple sensors lacking dedicated hardware acceleration.

1.2. Terms

To ensure clarity and precision throughout this report, it is essential to define the core technical terms and metrics used in the analysis of the proposed system. These terms form the basis for evaluating the cryptographic strength and operational efficiency of the implemented protocols.

- **Post-Quantum Cryptography (PQC):** Cryptographic algorithms, typically based on lattice, code, or multivariate quadratic problems, that are hypothesized to be secure against attacks by both quantum and classical computers.
- **Quantum Computing:** A field of computing based on the principles of quantum mechanics, utilizing qubits to perform complex calculations significantly faster than classical bits for specific algorithms.
- **Shor's Algorithm:** A quantum algorithm developed by Peter Shor that can find the prime factors of an integer in polynomial time, posing an existential threat to RSA and ECC[6].

- **Harvest Now, Decrypt Later (HNDL):** A strategy where adversaries intercept and store encrypted data today with the intention of decrypting it in the future using powerful quantum computers[7].
- **Lattice-based Cryptography:** A branch of post-quantum cryptography based on the hardness of mathematical problems in lattices, such as the Learning With Errors (LWE) problem, forming the basis for ML-KEM[8].
- **Module-Lattice-based Key Encapsulation Mechanism (ML-KEM):** A specific NIST-standardized KEM (formerly known as Kyber) that provides quantum-secure key exchange.
- **Key Encapsulation Mechanism (KEM):** A set of cryptographic functions used to establish a shared secret between two parties using asymmetric encryption without the risks associated with traditional key exchange.
- **Encapsulation / Decapsulation:** The processes in a KEM where the sender wraps a shared secret in a ciphertext (encapsulation) and the receiver unwraps it using their private key (decapsulation).
- **Public Key / Private Key:** The components of an asymmetric key pair; the public key is used for encryption or encapsulation, while the private key is kept secret and used for decryption or decapsulation.
- **Shared Secret:** A cryptographic key derived through a KEM, known only to the sender and receiver, used to encrypt subsequent data transfers using symmetric algorithms.
- **SHAKE256:** A cryptographic hash function and Extendable-Output Function (XOF) from the SHA-3 family, used in this project as a stream cipher for high-throughput nodes[9].
- **Quantum One-Time Pad (OTP):** A specialized implementation of the One-Time Pad concept where a unique, quantum-safe shared secret is used to encrypt a single data chunk for constrained devices.
- **Exclusive OR (XOR):** A logical bitwise operation used to combine plaintext with a key or keystream to produce ciphertext, providing perfect secrecy if the key is truly random and as long as the data.
- **Entropy:** A measure of the randomness or unpredictability of data; high entropy in keys is critical to preventing successful brute-force or pattern-based attacks[10].

- **Chi-square Distribution:** A statistical probability distribution used in randomness testing to determine the ”goodness of fit.” It measures how much the observed frequency of byte values deviates from the expected uniform distribution. A value within certain percentage bounds (e.g., 10% to 90%) indicates that the sequence is likely random[10].

- **Arithmetic Mean:** The average value calculated by summing all data points (byte values) and dividing by the total count. For a truly random sequence of 8-bit bytes, the arithmetic mean should ideally be close to 127.5[10].

- **Monte Carlo Value for Pi:** A method of estimating the mathematical constant π through random sampling. In entropy testing, random bytes are used to represent coordinates within a square; the ratio of points falling inside an inscribed circle to the total points allows for an approximation of π. The closer the result is to 3.14159..., the higher the quality of the randomness source[10].

- **Serial Correlation:** A statistical measure that identifies the relationship between a data point and its predecessor in a sequence. In cryptography, it evaluates whether a byte's value depends on the previous byte. A serial correlation coefficient close to zero indicates that the bytes are independent and unpredictable[10].

- **Plaintext / Ciphertext:** The original, readable data before encryption and the resulting unreadable data after the encryption process, respectively.

- **Replay Attack:** A type of active network attack where a valid data transmission is captured and maliciously re-sent to subvert the system's logic[2].

- **Man-in-the-Middle (MitM):** An attack where the adversary secretly intercepts and potentially alters the communication between two parties who believe they are communicating directly.

- **Timestamp Validation:** A security mechanism where a time-based value is embedded in a packet and verified by the receiver to ensure the freshness of the data and prevent replay attacks.

- **Forward Secrecy:** A security property that ensures a compromise of long-term keys does not compromise past session keys.

- **Session Key / Ephemeral Key:** A short-lived cryptographic key used for a single session or transaction, enhancing security by limiting the amount of data encrypted with one key.

- **Heterogeneous Network:** A network consisting of diverse devices with varying computational power, memory, and communication protocols, such as a mix of Raspberry Pi and ESP32 nodes.
- **Resource-constrained Device:** Hardware with limited processing power (CPU), memory (RAM), and energy, such as the ESP32 microcontroller used in this report.
- **Gateway:** A central node that acts as a bridge between different networks or protocols, managing traffic and security for connected IoT devices.
- **SCADA (Supervisory Control and Data Acquisition):** A system architecture used for high-level process supervisory management, often involving the control of industrial sensors and actuators.
- **NIST (National Institute of Standards and Technology):** The U.S. agency responsible for developing and maintaining cryptographic standards, including the PQC standardization project.
- **FIPS (Federal Information Processing Standards):** Publicly announced standards developed by the U.S. federal government for use in computer systems, such as FIPS 203 for ML-KEM[1].
- **Latency:** The time delay in a system, measured here as the time taken for a complete cryptographic transaction between a node and the gateway.
- **Throughput:** The rate at which data is successfully transmitted or processed by the system, often measured in bytes per second or packets per second.
- **Data Integrity:** The assurance that data has not been altered or corrupted during transmission, typically verified through hash functions or digital signatures.
- **Data Confidentiality:** The property that ensures information is not made available or disclosed to unauthorized individuals, entities, or processes.
- **Availability:** The assurance that authorized users have timely and reliable access to data and resources (the third pillar of the CIA triad).
- **Cryptographic Hash Function:** A mathematical algorithm that maps data of arbitrary size to a fixed-size bit string, used for integrity checks and key derivation.
- **Advanced Encryption Standard (AES):** A classical symmetric-key algorithm used as a benchmark for comparing the performance of post-quantum implementations in this report.

- **Elliptic Curve Cryptography (ECC):** A classical public-key cryptography approach based on the algebraic structure of elliptic curves, targeted by quantum algorithms.
- **Clock Drift:** The phenomenon where the internal clocks of different devices in a network become unsynchronized over time, affecting timestamp-based security mechanisms.
- **Protocol Stack:** The hierarchical set of network protocols that work together to provide communication services, such as TCP/IP or UDP/IP used in this study.

1.3. Problem Definition: The Quantum Threat to Critical Infrastructure

Modern industrial civilization depends heavily on Supervisory Control and Data Acquisition (SCADA) systems. These systems use millions of low-power sensors to monitor vital parameters such as pressure in gas pipelines, temperature in reactor cores, and voltage frequency in electrical grids. In the current situation, the confidentiality and integrity of these telemetry data rely entirely on the classical Public Key Infrastructure (PKI) [11]. This infrastructure is based on algorithms such as RSA and ECC that have been considered secure for decades; however, the mathematical assumptions ensuring the security of these algorithms are now being shaken with the advancing technology.

The security assumptions underlying these classical algorithms are collapsing. The emergence of Cryptographically Relevant Quantum Computers (CRQC) poses a threat to break the mathematical difficulty of factoring integers (on which RSA is based) and discrete logarithm problems (on which ECC is based). Shor's Algorithm, formulated by Peter Shor in 1994, has theoretically shown that a sufficiently powerful quantum computer can solve these problems in polynomial time, making existing encryption obsolete [6]. This situation leaves not only personal data but also state secrets and critical energy lines vulnerable to exploitation by quantum-enabled adversaries.

1.3.1. The "Harvest Now, Decrypt Later" Attack Vector

While the production of a CRQC capable of breaking 2048-bit RSA keys may be a decade away, critical infrastructures are faced with an immediate and retrospective threat known as "Harvest Now, Decrypt Later" (HNDL) [7]. This attack scenario follows these steps:

(i) Attackers intercept and store the encrypted traffic from critical sensors today.

(ii) This data remains encrypted and unreadable for years, but it is stored in digital archives.

(iii) Once a CRQC is developed, the attacker uses it to retrospectively decrypt the stored data.

For a credit card transaction, this situation may not pose a problem because the card will expire. However, for industrial data with a validity life of decades—for example, nuclear waste records, structural integrity data of dams, or sensitive military location data—HNDL poses an unacceptable strategic risk. This means that data transmitted today constitutes a threat for tomorrow.

1.4. Purpose and Scenario Description

The main purpose of this project is to demonstrate a practical, hardware-aware cryptographic architecture that enables legacy industrial sensors to communicate securely in a post-quantum era. Beyond an abstract implementation, the project simulates a specific **Critical Industrial Monitoring Scenario**. This scenario is based on the network structure of a real-world manufacturing plant or power plant. The aim of the system is to collect devices with different capabilities under a single security umbrella and provide centralized management for heterogeneous networks.

- **The Field Sensor (ESP32):** This component represents a legacy pressure sensor located in a restricted area of a power plant. It typically has limited processing power (in the MHz range), limited memory (in the KB range), and transmits vital, low-bandwidth telemetry data. The main challenge here is to ensure its security without replacing this device—which would be costly. ESP32 is a widely preferred node representative in the IIoT field due to its low energy nature.
- **The Edge Gateway (Raspberry Pi 5):** This represents a modern data collector or surveillance node within the same facility. It manages high-bandwidth traffic (such as video streams) and has sufficient computational resources for advanced cryptographic operations. Raspberry Pi 5 is a powerful platform demonstrating modern edge computing capabilities.
- **The Unified Gateway (Server):** It serves as the central control center and can manage heterogeneous connections simultaneously by decrypting data from both legacy and modern devices. This server works as the ”conductor” of the system and applies the

appropriate security layer to each device type by separating the traffic based on the port.

By validating this scenario, the project aims to prove that adaptable software architectures can bridge the gap between advanced cryptographic standards and constrained industrial hardware. The heterogeneous structure of the system shows that different security needs can be combined in a single ”Unified” structure.

1.5. Report Organization

This report is structured to guide the reader through the theoretical foundations, practical implementation, and verification of the engineering process. Each chapter deepens the topic by building on the previous one and detailing a different stage of the process:

- **Chapter 2 (Background):** Provides an in-depth look at the theoretical foundations of cryptography, quantum mechanics, and the hardware features of the devices used. In this section, the scientific ground on which the system is built and the reasons for choosing PQC algorithms are explained in an academic language.
- **Chapter 3 (Analysis and Methodology):** Details the threat models (Replay, MitM), analyzes the requirements, and justifies the algorithmic choices. While security analyses are put in a theoretical framework in this section, solution proposals are discussed over the mathematical models of attacks.
- **Chapter 4 (Design and Implementation):** Explains the software architecture, coding logic, and security protocols (Unified Architecture, Port Separation) in detail. The flowcharts in the design phase of the system and the code-level counterparts of the applied security layers are included in this section.
- **Chapter 5 (Test and Results):** Presents experimental data, performance metrics, and security verification results. The performance of the system in real-world scenarios is analyzed and presented through graphs and tables.
- **Chapter 6 (Conclusion):** Summarizes the project’s findings and suggests future research directions. The academic contributions and industrial application potential of the study are evaluated in this final section.

2. BACKGROUND

The technological landscape of this project sits at the intersection of three distinct yet deeply interconnected fields: Quantum Mechanics, Advanced Mathematics (specifically Lattice Theory), and Embedded Systems Engineering. To fully appreciate the architectural decisions, protocol designs, and implementation strategies detailed later in this report, one must first establish a rigorous understanding of the fundamental principles governing these domains. This chapter provides an exhaustive theoretical foundation designed to bridge the gap between abstract mathematical concepts and practical hardware constraints. It begins by examining the mathematical formalism of Quantum Computing to explain the underlying mechanisms that render classical RSA encryption vulnerable. Following this, it transitions to the Linear Algebra and geometry behind Lattice-based cryptography to provide a technical justification for why ML-KEM is capable of resisting quantum-enabled attacks. Finally, the chapter provides a deep-dive into the hardware-specific architectures of the ESP32 and Raspberry Pi 5, focusing on memory addressing, interrupt handling, and I/O constraints that directly influence the segmentation and chunking logic of our proposed implementation.

2.1. Cryptographic Primitives and Hardness Assumptions

Modern digital security is not built on absolute mathematical impossibility, but rather on the concept of ”Hard Problems.” These are mathematical tasks that are computationally trivial to verify when the solution is known but remain infeasible to solve within a reasonable timeframe—even for the world’s most powerful supercomputers—without specific secret information, commonly referred to as the cryptographic key. The robustness of a security system is therefore tied to the estimated time and energy required to perform a brute-force or algorithmic search for this key, a concept known as computational security.

2.1.1. Classical Public-Key Cryptography (PKC)

For the past four decades, the global infrastructure for secure communication, digital signatures, and identity verification has relied almost exclusively on two primary mathematical hardness problems. These problems are elegant in their simplicity but provide immense security against classical binary computation.

(i) **Integer Factorization (RSA):** The security of the RSA (Rivest-Shamir-Adleman) algorithm is founded on the difficulty of finding the prime factors of a very large com-

posite integer. Given an integer $N = p \cdot q$, where p and q are two large, distinct prime numbers, multiplying them to find N is a process that can be completed in $O(n^2)$ time. However, the inverse process—starting with N and attempting to derive p and q—is conjectured to be hard. Classical computers must rely on algorithms such as the General Number Field Sieve (GNFS), which has a sub-exponential time complexity. As key sizes increase to 2048 or 4096 bits, the time required for classical factorization grows beyond the age of the universe.

(ii) **Discrete Logarithm Problem (ECC):** Elliptic Curve Cryptography (ECC) utilizes the algebraic structure of elliptic curves over finite fields. Given a known generator point G on the curve and a public point $Q = k \cdot G$, the problem of finding the scalar integer k is known as the Discrete Logarithm Problem. Unlike integer factorization, no sub-exponential algorithm is known for solving this problem on classical computers, which is why ECC can offer the same level of security as RSA but with significantly smaller key sizes (e.g., 256-bit ECC is roughly equivalent to 3072-bit RSA). This efficiency has made ECC the dominant standard for resource-constrained IoT sensors and mobile devices.

2.2. Quantum Computing Fundamentals

The hardness assumptions that secure RSA and ECC hold true only when computation is performed on classical Turing machines. However, the emergence of quantum computers, which operate on fundamentally different physical and mathematical principles, necessitates a complete re-evaluation of these complexity classes. Quantum computing does not merely provide a faster version of classical processing; it enables a new form of information processing that utilizes the strange and counter-intuitive laws of quantum mechanics to perform tasks that are physically impossible for bits.

2.2.1. The Qubit and Superposition

The fundamental unit of quantum information is the **Qubit**. While a classical bit is always restricted to a state of either $|0\rangle$ or $|1\rangle$, a qubit can exist in a linear combination of both states simultaneously through a property known as superposition. This is not a simple ”halfway” state between 0 and 1, but a complex mathematical state where the qubit holds the potential for both outcomes until it is measured. Mathematically, a pure qubit state $|\psi\rangle$ is represented as a unit vector in a two-dimensional complex Hilbert space:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \tag{2.1}$$

In this equation, α and β are complex probability amplitudes. Upon measurement, the qubit will collapse into state $|0\rangle$ with probability $|\alpha|^2$ or state $|1\rangle$ with probability $|\beta|^2$, satisfying the normalization condition $|\alpha|^2 + |\beta|^2 = 1$. This ability to inhabit multiple states at once allows quantum systems to encode information in ways that exceed the capacity of classical registers.

2.2.2. Superposition and Parallelism

When multiple qubits are combined into a quantum register, the power of superposition scales exponentially. A system composed of N qubits exists in a coherent superposition of all 2^N possible basis states at the same time:

$$|\Psi\rangle = \sum_{x=0}^{2^N-1} c_x|x\rangle \tag{2.2}$$

While a classical N-bit register can hold only one value out of 2^N at any given moment, a quantum register holds them all. This allows quantum algorithms to perform a single computational operation that affects all 2^N inputs simultaneously. This phenomenon, known as quantum parallelism, allows a quantum computer to explore vast solution spaces in a single step, providing a level of parallelism that classical supercomputers, regardless of their size or speed, cannot replicate.

2.2.3. Quantum Gates and Entanglement

The manipulation of qubits to perform meaningful computation is achieved through the use of Quantum Gates. Unlike classical logic gates such as AND, OR, or XOR, which are often irreversible, quantum gates are represented by reversible unitary matrices. The Hadamard Gate (H) is perhaps the most critical, as it is used to put a qubit into a state of superposition. However, the true power of quantum computing is unlocked through the CNOT Gate, which is used to create Entanglement.

Entanglement is a unique quantum phenomenon where the states of two or more qubits become inextricably linked. When qubits are entangled, the state of one qubit becomes instantly correlated with the state of another, regardless of the physical distance between them. This allows a quantum algorithm to perform operations on the global state of the entire system rather than processing individual bits in isolation. By carefully orchestrating entanglement and superposition, quantum computers can interfere with the probability amplitudes of different outcomes—constructively for the correct answer and destructively for the wrong ones—enabling computational speedups that are physically impossible for classical Turing machines.

2.3. Cryptographic Hardness Problems

As previously established, modern cryptography relies on ”Hard Problems”—mathematical tasks that are designed to be easy to verify but computationally infeasible to solve without the correct secret key. To understand the transition to Post-Quantum Cryptography, we must first analyze the specific mechanisms by which quantum computers dismantle these traditional hardness assumptions.

2.3.1. The Vulnerability: Integer Factorization and DLP

Current Industrial IoT standards, including the widely used TLS/SSL and IPsec protocols, rely on the security of the two main problems discussed earlier: Integer Factorization for RSA-based systems and the Discrete Logarithm Problem (DLP) for ECC-based systems. These algorithms form the ”Chain of Trust” for everything from firmware updates to secure SCADA telemetry. In the classical domain, these problems are secure because the fastest known algorithms still take exponential time relative to the key size. However, the introduction of a quantum processor changes the underlying mathematical rules of the game, turning what was once a wall into a door.

2.3.2. Shor's Algorithm: The Mathematical Weapon

In 1994, Peter Shor formulated a quantum algorithm that demonstrated that both Integer Factorization and the Discrete Logarithm Problem could be solved in polynomial time. This was a revolutionary discovery because it moved these problems from the "Hard" (NP-Hard) category to the "Easy" (P) category within the context of quantum complexity classes. Shor's algorithm relies on the Quantum Fourier Transform (QFT) to find the "period" of a modular exponentiation function, which is the key step required to factor large numbers or find discrete logs.

The existential threat to classical cryptography can be summarized by comparing the time complexities:

$$\text{Time}Classical \approx O(e^{1.9(\log N)^{1/3}(\log\log N)^{2/3}}) \quad \text{(Exponential)} \tag{2.3}$$

$$\text{Time}Quantum \approx O((\log N)^3) \quad \text{(Polynomial)} \tag{2.4}$$

While a 2048-bit RSA key would take trillions of years for a classical supercomputer to crack, a sufficiently large quantum computer running Shor's algorithm could potentially do it in hours or even minutes. This effectively destroys the mathematical foundations of current industrial security, necessitating a shift to algorithms that Shor's algorithm cannot simplify.

2.4. Post-Quantum Cryptography: Lattice Theory

To build systems that remain resilient in the age of quantum computing, we must identify mathematical structures that are resistant to the Quantum Fourier Transform and Shor's algorithm. The most robust and versatile candidate for this task is Lattice-Based Cryptography[12]. These schemes are not only theoretically secure against quantum attacks but are also efficient enough to be implemented on the diverse range of hardware found in the IoT ecosystem.

2.4.1. Lattices in Euclidean Space

A lattice $\mathcal{L}$ is defined as a discrete set of points in n-dimensional Euclidean space that possesses a periodic, grid-like structure. It is generated by a set of linearly independent basis vectors $\mathbf{B} = \{\mathbf{b}_1, \ldots, \mathbf{b}_n\}$. Every point in the lattice is formed by a linear combination of these basis vectors using only integer coefficients. While simple to visualize in two or three dimensions, lattices used in cryptography often operate in hundreds or thousands of dimensions.The security of lattice-based schemes is rooted in the Shortest Vector Problem (SVP). Given a complex basis for a high-dimensional lattice, the task is to find the shortest non-zero vector in that lattice. While this is trivial in low dimensions, finding the shortest vector becomes computationally ”NP-Hard” as the number of dimensions increases. Critically, even quantum computers do not have a known efficient algorithm to solve SVP or its variants in high dimensions (typically $n > 500$)[8], making it an ideal foundation for quantum-safe security.

2.4.2. The Learning With Errors (LWE) Problem

The specific algorithm selected for this project, ML-KEM (Kyber), relies on a variant of the SVP known as the Module Learning With Errors (ML-WE) problem. Unlike classical schemes that rely on the properties of prime numbers, ML-WE involves solving a system of linear equations that has been intentionally corrupted with ”noise.” The fundamental equation is:

$$\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \pmod{q} \tag{2.5}$$

In this framework, $\mathbf{A}$ is a public matrix, $\mathbf{s}$ is the secret vector, and $\mathbf{e}$ is a small error vector (noise) drawn from a specific probability distribution. Without the error term, $\mathbf{s}$ could easily be found using standard Gaussian Elimination. However, the presence of the noise $\mathbf{e}$ makes it computationally impossible to recover the secret without knowing exactly how the noise was generated. Because Shor’s algorithm is designed to find periodic structures in numbers and not to solve noisy linear systems in high-dimensional space, lattice-based cryptography remains secure where RSA and ECC fail. This mathematical resilience is the primary reason for choosing ML-KEM-768 for our industrial security gateway.

2.5. Standards and Regulations

The transition from classical to post-quantum cryptography is a global effort governed by rigorous standardization processes and industrial regulations. This ensures that new cryptographic primitives are not only mathematically sound but also interoperable across different platforms and vendors.

2.5.1. NIST PQC Standardization Process

Recognizing the impending quantum threat, the National Institute of Standards and Technology (NIST) initiated a multi-year global competition in 2016 to identify and standardize post-quantum algorithms. This process involved some of the world's leading cryptographers and underwent several rounds of intense public scrutiny and cryptanalysis.

- **Round 1 (2017):** 69 initial candidates were submitted, covering various mathematical families including lattice-based, code-based, and multivariate systems.
- **Round 3 (2020):** Finalists were selected based on their security, performance, and key sizes. Lattice-based candidates emerged as the clear leaders for general-purpose encryption.
- **Selection (2022):** CRYSTALS-Kyber was selected as the primary standard for Key Encapsulation Mechanisms (KEM) due to its exceptional balance of high-speed performance and relatively small key and ciphertext sizes.
- **FIPS 203 (2024):** Kyber was formally standardized as ML-KEM[1]. This project strictly adheres to the technical specifications outlined in the FIPS 203[1] standard to ensure industrial compliance and security longevity.

2.5.2. Industrial Standards: IEC 62443

In the specific domain of Industrial IoT and SCADA systems, security is governed by the IEC 62443 standard, titled "Security for Industrial Automation and Control Systems"[13]. This regulation defines the security levels required to protect critical infrastructure from different tiers of adversaries.

- **Security Level 3 (SL-3):** Our project aims to meet SL-3 requirements, which mandate protection against intentional violations by sophisticated adversaries using professional tools and moderate resources.
- **Data Confidentiality and Integrity:** The standard strictly requires that all sensitive data in transit, especially control commands and high-precision telemetry, must be encrypted and protected against tampering.

By implementing post-quantum primitives within a unified gateway architecture, this project provides a path for existing industrial systems to transition to IEC 62443 compliance while future-proofing against the quantum threat.

2.6. Hardware Architecture Analysis

The feasibility of implementing PQC in an industrial environment is ultimately determined by the physical hardware. A deep understanding of the internal architectures of our target devices—the ESP32 and the Raspberry Pi 5—is essential for explaining the ”Segmentation” and ”Chunking” logic that defines our system’s implementation.

2.6.1. ESP32-WROOM-32 Internal Architecture

The ESP32 is a powerful System-on-Chip (SoC) designed specifically for low-power, high-integration IoT applications[14]. However, its design presents unique challenges for memory-intensive PQC algorithms.

- **CPU Core:** It features an Xtensa® Dual-Core 32-bit LX6 microprocessor, capable of running up to 240 MHz. While computationally fast, the 32-bit architecture limits the efficiency of large-integer arithmetic common in some PQC schemes.
- **Memory Map (The Critical Bottleneck):** Although the device datasheet mentions 520 KB of internal SRAM, this memory is not a single, contiguous block available to the programmer. It is divided into specialized regions:
  - **IRAM (Instruction RAM):** 192 KB dedicated to code execution and time-critical interrupt routines.
  - **DRAM (Data RAM):** Approximately 328 KB used for variables, stack, and the heap.

The Fragmentation Issue: The ML-KEM-768 algorithm requires the allocation of large, continuous arrays to perform polynomial multiplications (specifically the PolyVec structures). In a real-world environment where the TCP/IP stack (LwIP)[15], Wi-Fi drivers, and RTOS overhead are already consuming memory, the DRAM becomes highly fragmented. Attempting to allocate a contiguous 50KB block often fails even if the total free memory is higher. This physical limitation was the primary driver for the ”Chunking” protocol developed in this project, which breaks the telemetry data into 32-byte segments to fit within the ESP32’s fragmented memory pool.

2.6.2. Raspberry Pi 5 Architecture

In contrast to the constrained ESP32, the Raspberry Pi 5 acts as our Edge Gateway, representing a high-end compute node capable of significantly more intensive cryptographic tasks.

- **SoC:** It is powered by the Broadcom BCM2712, a quad-core 64-bit Arm Cortex-A76 processor running at 2.4GHz. This 64-bit architecture is naturally suited for the large array manipulations required by PQC.
- **Operating System and Memory Management:** Running a full Linux distribution (Debian), the Pi 5 utilizes advanced virtual memory management and paging.

Unlike the ESP32, which operates on physical memory addresses and suffers from fragmentation, the Raspberry Pi can dynamically allocate large chunks of memory for PQC public keys (1184 bytes) and ciphertexts (1088 bytes) without needing specialized segmentation logic. This stark contrast in hardware capability defines the ”Heterogeneous” nature of our network model and justifies the use of the high-throughput SHAKE256 protocol on this specific communication lane.

3. ANALYSIS AND METHODOLOGY

This chapter bridges the gap between the theoretical threats identified in the Introduction and the practical implementation detailed in the Design chapter. Before a single line of code is written for the Unified Gateway or the IoT nodes, a rigorous engineering analysis is required to ensure that the resulting system is not only secure but also operationally viable within the rigid constraints of a real-world industrial environment. This chapter defines the specific ”Threat Model” relevant to Industrial IoT (IIoT), providing a granular explanation of active network attacks such as Man-in-the-Middle (MitM) and Replay attacks, which are particularly devastating in SCADA (Supervisory Control and Data Acquisition) contexts. It then transitions into a comprehensive analysis of the requirements for a Post-Quantum solution, establishing the criteria for success in terms of both security and performance. A significant portion of this chapter is dedicated to the justification for selecting the ML-KEM algorithm over other PQC candidates, such as code-based or multivariate schemes. Finally, it presents the core methodology behind the ”Quantum One-Time Pad” protocol, detailing the mathematical reasoning and the architectural logic used to provide maximum confidentiality on resource-constrained devices without triggering memory overflows or watchdog resets.

3.1. System Requirements Specification

Before proceeding to the architectural design phase, it is imperative to define the specific engineering requirements that the system must satisfy to be considered successful. These requirements are derived from an intersection of the physical constraints inherent in Critical Infrastructure (CI) environments—where uptime and safety are paramount—and the mathematical threat model posed by the advent of Cryptographically Relevant Quantum Computers (CRQCs). By clearly articulating these specifications, we establish a benchmark against which the final implementation can be measured during the testing phase.

3.1.1. Functional Requirements (FR)

Functional requirements define the specific behaviors, security functions, and services that the Unified PQC system must support. These requirements ensure that the system addresses the ”Quantum Gap” in current IoT security without sacrificing the interoperability needed for diverse industrial nodes.

- **FR-01 Quantum Resistance:** The system MUST utilize cryptographic algorithms officially standardized by NIST (FIPS 203 / ML-KEM-768)[1]. This ensures that the mathematical difficulty of the encryption is based on the Shortest Vector Problem (SVP) in high-dimensional lattices, which is currently resistant to both Shor's and Grover's algorithms.
- **FR-02 Perfect Forward Secrecy (PFS):** To mitigate the "Harvest Now, Decrypt Later" threat, the system MUST generate a new, ephemeral session key for every distinct data segment or transaction. By ensuring that keys are refreshed frequently, the system guarantees that the compromise of a single key does not compromise past or future telemetry data, thereby isolating the impact of any potential breach.
- **FR-03 Heterogeneous Support:** The Unified Gateway MUST be architecturally capable of simultaneously managing concurrent connections from diverse hardware tiers. This includes resource-constrained microcontrollers (ESP32) that require optimized, low-overhead protocols, and high-performance edge nodes (Raspberry Pi 5) capable of handling more intensive stream ciphers.
- **FR-04 Replay Attack Prevention:** The protocol MUST include a robust mechanism to ensure the "freshness" of every packet. This is achieved through the integration of 64-bit Unix Timestamps within the encrypted payload, allowing the Gateway to detect and reject any re-transmitted valid packets that have been captured by an adversary.
- **FR-05 Device Authenticity and Trust:** The system MUST ensure that legacy sensors communicate only with a verified and trusted Gateway. This is implemented through a pre-shared trust model where the Gateway's PQC public key is hardcoded into the sensor firmware, preventing an attacker from redirecting telemetry to a rogue server.

3.1.2. Non-Functional Requirements (NFR)

Non-functional requirements specify the operational criteria and quality attributes used to evaluate the performance and reliability of the system, rather than its specific features. In an industrial context, these requirements are often as critical as the security functions themselves, as a secure system that causes a control loop failure is unusable.

Table 3.1. Non-Functional Requirements for Industrial IoT PQC

ID	Requirement Description	Target Metric
NFR-01	**Latency:** The additional cryptographic overhead introduced by ML-KEM[1] handshakes and encryption must not cause the total round-trip telemetry transmission time to exceed the tolerance of real-time SCADA control loops.	< 100 ms per packet
NFR-02	**Entropy Quality:** The random keys generated by the Gateway and the shared secrets derived on the nodes must exhibit high statistical randomness to ensure the integrity of the One-Time Pad property.	> 7.9 bits/byte
NFR-03	**Reliability:** The system must operate continuously in an industrial environment without suffering from memory leaks, fragmentation-induced crashes, or watchdog timer resets on the constrained microcontrollers.	99.9% Uptime
NFR-04	**Scalability:** The Unified Gateway should be able to handle high-bandwidth loads, such as concurrent data streams and log uploads, without exceeding sustainable CPU saturation limits.	$< 20\%$ CPU Usage
NFR-05	**Code Size:** The compiled firmware implementation, including the PQC libraries and the network stack, must fit within the available Flash memory of legacy IoT devices.	< 4 MB

The target metrics derived from these safety standards are summarized in Table 3.1. The selection of these specific metrics and their corresponding target values is deeply rooted in established industrial safety and cryptographic standards. For instance, the latency target of less than 100 ms (NFR-01) is derived from the operational requirements of Supervisory Control and Data Acquisition (SCADA) systems, where control loops for critical processes—such as pressure regulation or automated power grid frequency adjustments—demand deterministic response times to ensure system stability and physical safety. According to industrial security frameworks like IEC 62443 [13], excessive delay in telemetry transmission can lead to desynchronization in real-time control, potentially resulting in catastrophic equipment failure. Therefore, the sub-100 ms threshold serves as a mandatory ceiling to ensure that the

added Post-Quantum overhead does not compromise the functional safety of the industrial process.

Similarly, the entropy and reliability targets (NFR-02 and NFR-03) are selected to satisfy the mathematical rigor required for information-theoretic security in a quantum era. The entropy target of greater than 7.9 bits per byte is a direct application of NIST SP 800-22 [10] recommendations for high-quality random number generation, which is the foundational requirement for the proposed ”Quantum One-Time Pad” protocol. Following Shannon’s theory [16], the security of a one-time pad is only absolute if the pad is truly random and never reused; hence, any statistical bias would introduce a vulnerability that a quantum adversary could exploit through pattern analysis. Furthermore, the 99.9% uptime requirement reflects the ”Availability” pillar of the industrial CIA triad, ensuring that the memory-intensive cryptographic tasks performed by the ESP32 do not lead to resource exhaustion or watchdog-induced reboots that would disrupt critical monitoring services.

3.2. Threat Modeling for Industrial Control Systems

In the field of cybersecurity, a threat model is a structured representation of all the potential vulnerabilities and attack vectors that affect a specific system. For a SCADA or IIoT system, the stakes are not merely digital; they are physical and environmental. A successful attack on an industrial network can result in the loss of life, environmental catastrophe, or the destruction of critical machinery (e.g., a pipeline exploding because a sensor was tricked into reporting false pressure data).

3.2.1. Man-in-the-Middle (MitM) Attack Analysis

Definition and Mechanics: A Man-in-the-Middle (MitM) attack occurs when an adversary secretly intercepts, relays, and potentially alters the communication between two parties who believe they are communicating directly with each other through a trusted channel [2]. In a quantum-vulnerable network, this is often achieved by exploiting the lack of mutual authentication or the ease of breaking the underlying classical exchange protocol.

Scenario in Industrial IoT: Consider an attacker who has gained access to the local industrial Wi-Fi network. They place a rogue device or use ARP poisoning to position themselves between the ESP32 field sensor and the Wi-Fi Access Point. The sensor transmits an encrypted packet containing ”Pressure: 50 Bar,” which is a dangerous level. The attacker intercepts this packet, and although they may not be able to decrypt it without a quantum

computer (if PQC is used), they can attempt to drop the packet or alter the unencrypted headers to disrupt the flow. In a classical ECC/RSA system, the attacker could eventually use a quantum computer to solve the discrete log problem, derive the session key, change the value to ”Pressure: 10 Bar” (to hide a dangerous buildup from the operators), and forward the modified packet to the server.

Proposed Mitigation: In this project, MitM is neutralized through the cryptographic ”binding” provided by the Key Encapsulation Mechanism (KEM). The initial handshake establishes a 256-bit shared secret using ML-KEM-768, a process that is mathematically resistant to interception. Furthermore, the firmware of the IoT devices utilizes a pre-shared trust model where the Gateway’s identity is verified through hardcoded public keys during the initial encapsulation phase. This prevents an attacker from successfully masquerading as the Gateway to the field sensors. The interaction between the legitimate nodes and the potential attacker during such an intervention is visually modeled in Figure 4.2 within the Chapter 4.

3.2.2. Replay Attack Analysis

Definition and Mechanics: A Replay Attack involves the passive interception of a valid data transmission followed by its subsequent re-transmission by an adversary at a later time. Because the packet itself is valid and correctly encrypted, a simple cryptographic check will not detect the attack unless a freshness mechanism is in place.

Scenario in Industrial IoT: Imagine a scenario where the ESP32 sensor is used to send a specific command: ”Open Safety Valve.” An attacker records this encrypted packet as it traverses the network. Even if the packet is encrypted with quantum-safe algorithms, the attacker does not need to decrypt it to cause damage. At a strategically chosen time, such as during a shift change, the attacker re-sends the exact same binary recording to the server. The server, seeing a validly encrypted packet, decrypts it and executes the ”Open Safety Valve” command again, potentially causing a chemical spill or pressure drop.

Proposed Mitigation: To prevent this, our methodology mandates the enforcement of ”Temporal Freshness.”

- **Timestamping:** Every data payload generated by the IoT node is prepended with a 64-bit Unix Timestamp (with microsecond precision) *before* the encryption process begins. This ensures that the time of packet creation is cryptographically bound to the data itself.

- **Server Logic and State Management:** Upon successful decryption, the Gateway extracts the timestamp and compares it to a stateful record of the ”last seen timestamp” for that specific device ID. If $T_{packet} \leq T_{last_seen}$, or if the timestamp is outside a predefined drift window, the packet is immediately discarded as a replay attempt. This ensures that a captured packet is only valid for a single, immediate execution.

The defensive boundary against this specific capture-and-resend strategy is illustrated in the system’s security use cases in Figure 4.2.

3.3. Methodology: The ”Quantum One-Time Pad”

Given the hardware constraints analyzed in Section 2.4, a standard TLS 1.3 implementation with PQC extensions is often impossible for microcontrollers like the ESP32 due to the massive RAM requirements for holding full certificates and large ciphertexts. Therefore, we propose a custom methodology that adapts the classic concept of the One-Time Pad (OTP) for the modern quantum-secure era. This methodology is designed to provide maximum security with minimal computational overhead on the sensor side.

- **Classical OTP Limitations:** A traditional OTP requires a pre-shared key (the pad) that is at least as long as the message being sent. While perfectly secure, this is impractical for IoT because the devices would need to store gigabytes of random keys, creating a massive storage and management overhead.
- **Proposed Quantum OTP:** We bridge this gap by using ML-KEM-768 to dynamically generate a 32-byte ”Shared Secret” for every 32-byte ”Chunk” of telemetry data. Instead of storing a pad, the device ”negotiates” a new pad on-the-fly for every transaction.
- **Mathematical Justification:** The output of the ML-KEM decapsulation process is a shared secret that is mathematically indistinguishable from random noise (exhibiting high entropy). By performing a bitwise Exclusive-OR (XOR) between this secret and the plaintext telemetry, we achieve information-theoretic security for that specific segment. Since the ”Pad” is refreshed for every chunk and never reused, it satisfies Claude Shannon’s criteria for perfect secrecy.

3.3.1. Theoretical Basis

The One-Time Pad (OTP) remains the only cryptosystem proven to have ”perfect secrecy” [16]. In information theory, perfect secrecy means that the ciphertext provides absolutely no information about the plaintext to an adversary, regardless of their computational power (even with an infinite quantum computer). This property is maintained only if the key meets three strict criteria:

(i) The key must be at least as long as the message being encrypted.

(ii) The key must be truly random (high entropy).

(iii) The key must be used exactly once and then destroyed.

3.3.2. Adaptation for IoT

Our methodology adapts these rigorous mathematical requirements for the fragmented memory environment of the ESP32:

- **Key Generation and Refreshing:** Instead of relying on a static pre-shared pad, we use the ML-KEM-768 algorithm to perform a new ”Encapsulation/Decapsulation” cycle for every message. This generates a dynamic, quantum-secure Shared Secret (SS) that acts as our ephemeral pad.
- **Segmentation Strategy:** To avoid memory allocation errors, the sensor data (including the timestamp) is broken into 32-byte chunks. Each chunk is processed independently.
- **Lightweight Encryption:** The encryption itself is reduced to a simple bitwise operation: $Ciphertext = Plaintext \oplus SS$. On the Gateway side, the process is reversed: $Plaintext = Ciphertext \oplus SS$.

Because the SS is derived from a lattice-based algorithm that is secure against Shor’s algorithm, and because the XOR operation is computationally trivial for a 240MHz microcontroller, this methodology allows us to maintain quantum-safe confidentiality without the latency or memory overhead of full-packet PQC-TLS. This effectively trades network bandwidth (due to the 1088-byte ciphertext overhead per 32-byte key) for extreme security and computational simplicity.

4. DESIGN AND IMPLEMENTATION

Having established the theoretical framework and analyzed the stringent hardware constraints in the previous chapters, this chapter details the comprehensive technical realization of the proposed solution. It transitions from the conceptual ”what” and ”why” to the practical ”how.” Here, we dissect the multi-layered software architecture, detailing the specific engineering decisions made to overcome the severe memory fragmentation of the ESP32 and the inherent latency challenges of Post-Quantum Cryptography (PQC). This chapter is structured to explain the system’s ”Dual-Lane” architecture, which provides a flexible framework for heterogeneous nodes. We explore the specific implementation of the ”Quantum One-Time Pad” protocol on the microcontroller, where the ESP32’s unique memory layout is optimized for high-security transactions, and the ”Stream Cipher” integration on the gateway for high-throughput data processing. Furthermore, it explicitly details the design-level security controls implemented to neutralize Man-in-the-Middle (MitM) and Replay attacks, ensuring that security is not a separate layer but is baked into the fundamental code logic and packet structures of the system.

4.1. Adaptive Software Architecture

The core philosophy of this project is ”Adaptability” through architectural intelligence. In a real-world Industrial IoT (IIoT) environment, a ”one-size-fits-all” security protocol is destined to fail because the hardware landscape is inherently heterogeneous. A low-power temperature sensor (ESP32) operating in a remote substation and a high-definition video surveillance node (Raspberry Pi) in a centralized control room have vastly different computational resources, power profiles, and throughput requirements.

To address this disparity, we designed a Dual-Lane Server Architecture that decouples the security logic from the hardware platform. The Unified Gateway (Server) does not treat all incoming connections with a uniform cryptographic template. Instead, it exposes two distinct logical interfaces, each governed by its own protocol stack and memory management policy:

(i) **Lane A (Port 65432):** This lane is optimized for high-bandwidth, continuous data streams. It utilizes the SHAKE256 algorithm as an extendable-output function to generate a fast, quantum-safe keystream. This lane is dedicated to powerful edge gateways and multi-sensor hubs like the Raspberry Pi, where data volume takes precedence over per-packet handshakes.

(ii) **Lane B (Port 65433):** This lane is optimized for low-bandwidth, high-security bursty telemetry. It employs our specialized Segmentation Protocol (Quantum One-Time Pad). This is dedicated to constrained microcontrollers like the ESP32, which often lack the RAM to maintain long-term encrypted sessions but must ensure absolute confidentiality for critical sensor readings.

This architectural decision eliminates the need for a complex, heavy handshake negotiation protocol (such as those found in TLS 1.3), which would otherwise consume precious CPU cycles and memory. The server knows exactly which cryptographic primitive to apply based solely on the destination port of the incoming TCP/IP packet, reducing the ”Time-to-First-Byte” latency significantly and ensuring that the gateway remains responsive even under heavy load. The high-level interaction between the heterogeneous nodes and the unified gateway, illustrating the port-based separation of the two distinct cryptographic lanes, is presented in Figure 4.1.

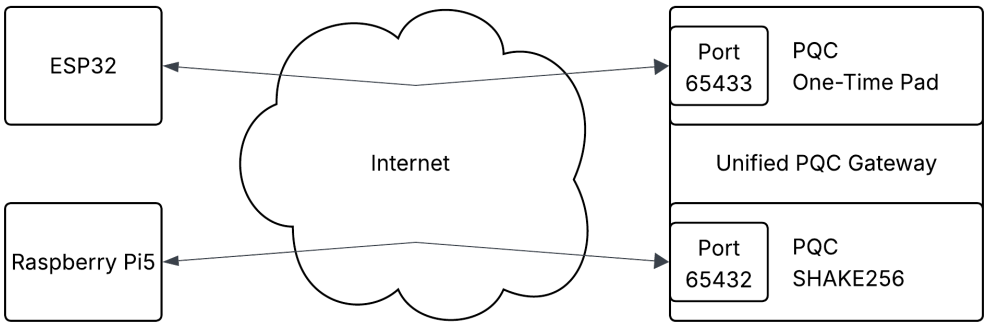

Figure 4.1. System Architecture Overview showing Dual-Lane Strategy

4.2. Finite State Machine (FSM) Logic

The stability and reliability of the ESP32 firmware in a production environment rely on a deterministic Finite State Machine (FSM). Unlike traditional procedural code that merely ”loops,” our implementation ensures that the device transitions between distinct, isolated states based on network health, cryptographic validation, and sensor events. This prevents ”race conditions” where the device might attempt to encrypt data before a valid key is generated.

(i) **STATE_INIT:** Upon power-on, the device initializes the Hardware Random Number Generator (HRNG)[17] to ensure high-entropy seeds. It then loads the Server's Public Key from a secure, immutable region of the flash storage (SPIFFS/NVS).

(ii) **STATE_WIFI_CONNECT:** The device attempts to authenticate with the Industrial Access Point. If the connection fails after a set number of retries, it enters a "Deep Sleep" back-off mode to prevent energy depletion and network congestion.

(iii) **STATE_READY:** The device samples the physical sensors via GPIO or I2C. This state includes a validation check to ensure the sensor data is within expected physical bounds.

(iv) **STATE_CRYPTO_OP:** This is the most resource-intensive state, divided into critical sub-tasks to monitor memory usage:

- Sub-state A: Generate local randomness for the KEM encapsulation.
- Sub-state B: Perform complex lattice-based matrix multiplications (ML-KEM Encapsulation).
- Sub-state C: Generate the XOR keystream by processing the derived shared secret.

(v) **STATE_TX:** The TCP socket is opened, the 1120-byte LQSP packet is flushed to the network buffer, and the socket is immediately closed to free up the heap for the next cycle.

(vi) **STATE_ERROR:** If any cryptographic check fails or a memory allocation error occurs, the device logs the error to a non-volatile register for post-mortem analysis and initiates a soft reboot.

4.3. System Interaction and Threat Defense Design

In our implementation, security is treated as a property of the protocol's robustness rather than a simple encryption layer. The design explicitly addresses the two most critical threats identified in the threat modeling phase: Replay Attacks and Man-in-the-Middle (MitM) attacks, by integrating validation logic directly into the packet processing pipeline. The functional roles of the system actors and the specific security defenses integrated into the architecture are detailed in the use case diagram shown in Figure 4.2. Specifically, the diagram illustrates how an adversary's 'Replay Attack Attempt' is effectively mitigated by the system's 'Block Invalid Packet' action, which is triggered if the stateful timestamp validation fails during the decryption phase.

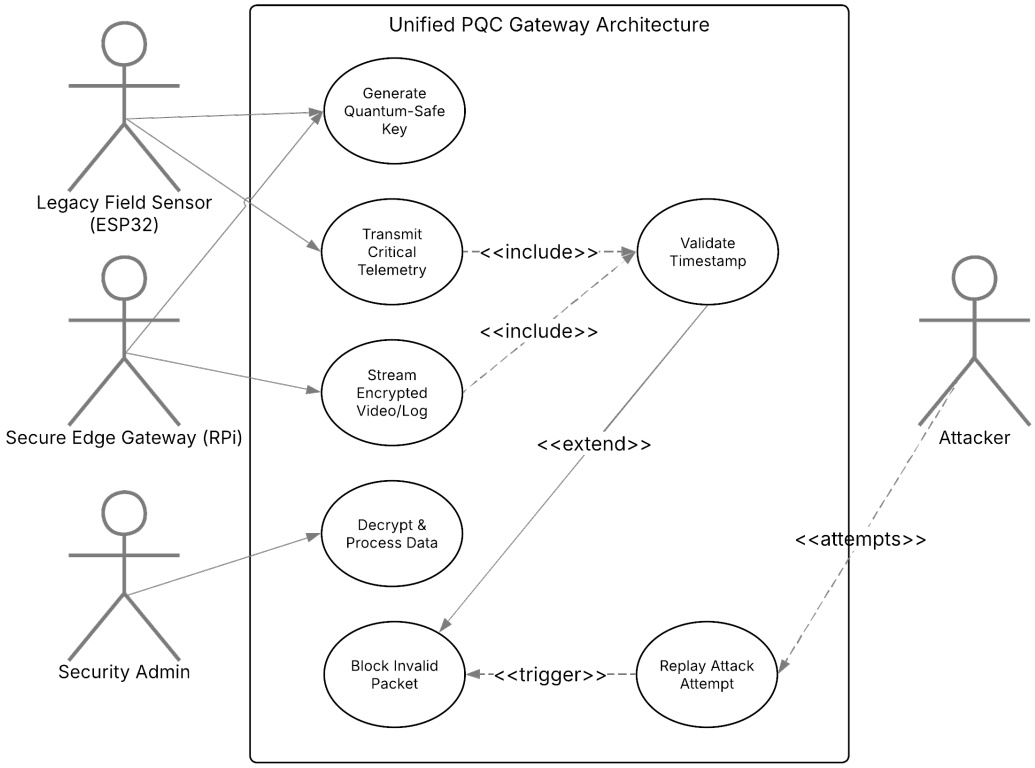

Figure 4.2. Use Case Diagram: System Actors and Security Defenses

4.3.1. Designing Against Replay Attacks

As analyzed in Chapter 3, a Replay Attack involves an adversary capturing a valid encrypted packet and re-transmitting it to trigger a redundant action. To prevent this at the design level, we implemented a strict Timestamp-Based Verification Mechanism that ensures ”Temporal Freshness.”

- **Client Side (ESP32):** Before the encryption phase, the firmware retrieves the current Unix Epoch time via the Network Time Protocol (NTP)[18]. This 64-bit integer is prepended to the plaintext sensor data.
- **Encryption Scope:** The internal payload structure is defined in a concatenated format as shown below:

$$P' = [Data \parallel Timestamp \parallel Padding]$$

The entire structure P' is then XOR-encrypted using the shared secret. Because the

timestamp is part of the encrypted payload, it is mathematically hidden from an attacker.

- **Server Side Logic:** Upon decryption, the server extracts $T_{current}$. It performs a stateful comparison against T_{last} (the most recent valid timestamp stored in the database for that specific device).
- **Rejection Condition:** The gateway code enforces a logic where if $T_{current} \leq T_{last}$, the packet is immediately flagged as a replay attempt. This renders any captured packet useless for future playback, even if the encryption itself remains unbroken.

4.3.2. Designing Against Man-in-the-Middle (MitM)

To prevent a sophisticated attacker from positioning themselves between the sensor and the gateway to intercept or alter keys, we implemented a Pre-Shared Trust (PST) Model.

- Unlike general web traffic where dynamic Certificates (PKI) are required for unknown servers, industrial sensors communicate with a fixed, known control center. This allowed us to simplify the chain of trust.
- We embedded the Gateway's ML-KEM Public Key (PK_{server}) directly into the immutable flash memory of the ESP32 clients during the provisioning phase.
- **Impact:** When the ESP32 performs an encapsulation to establish a shared secret, it uses this hardcoded, trusted public key. An attacker attempting to intercept the traffic cannot pose as the server because they do not possess the corresponding Private Key (SK_{server}), which is stored in the gateway's isolated secure memory. This ensures that only the intended gateway can decapsulate the secret and read the telemetry.

4.4. Legacy Sensor Implementation (ESP32)

Implementing NIST-standardized Post-Quantum Cryptography on a microcontroller with only 520KB of fragmented RAM represents the primary engineering challenge of this implementation. The following sections detail the memory-aware strategies used to make this possible.

4.4.1. Memory Management and Task Isolation

The standard Arduino or FreeRTOS environment on the ESP32 typically allocates a stack size of 4KB to 8KB[19] for the main execution loop. However, the ML-KEM-768 algorithm involves operations on large polynomial matrices (256×3 or 256×4 coefficients) that require significantly more temporary storage. Initial attempts to run the PQC logic within the main loop resulted in catastrophic *Stack Overflow* errors and system reboots.To resolve this, we implemented a manual memory management strategy by bypassing the default loop and creating a dedicated FreeRTOS task named `fullPqcTask`.

- **Stack Allocation:** We manually allocated 32KB of contiguous stack memory to this task, ensuring enough headroom for lattice operations.
- **Core Pinning:** The ESP32 is a dual-core SoC. We pinned the `fullPqcTask` to Core 1, while keeping Core 0 dedicated to the Wi-Fi stack and LwIP networking. This isolation prevents the intensive cryptographic calculations from blocking the network ”keep-alive” interrupts, maintaining a stable TCP connection during the 30-50ms encryption window.

4.4.2. The ”Chunking” Algorithm (Quantum One-Time Pad)

Since the ESP32 cannot encrypt a large multi-kilobyte telemetry message in a single pass due to its fragmented RAM, we developed a custom segmentation logic. This implementation is the practical realization of the ”Quantum One-Time Pad.”

Algorithm 1 ESP32 Quantum One-Time Pad Logic

Input: Large Telemetry Message M
Constant: Server Public Key PK_{server}
Chunk Size: 32 Bytes
for each chunk C_i in M **do**
  $SS, CT \leftarrow$ crypto_kem_enc(PK_{server}) {Generate Quantum Safe Key}
  $T \leftarrow$ get_timestamp()
  $Payload \leftarrow C_i + T$
  $Encrypted \leftarrow Payload \oplus SS$ {XOR Operation}
  send_to_network(CT) {Send Key Capsule (1088 Bytes)}
  send_to_network$(Encrypted)$ {Send Data}
end for

As detailed in Algorithm 1, for *every single 32-byte chunk* of data, the device performs a fresh ML-KEM Encapsulation. This produces a unique, 256-bit shared secret (SS) that acts as the ”Pad.” This secret is used once for the XOR operation and then immediately overwritten in memory. This strict ”Key Refreshing” cycle ensures that the system aligns with Shannon’s criteria for Perfect Secrecy [16], providing a level of confidentiality that is theoretically unbreakable by any future quantum computer.

4.4.3. Protocol Flow and Timing

The sequence of operations and the timing of the cryptographic handshake are visualized in Figure 4.3. As shown in the diagram, the Unified Gateway incorporates an ’Alternative’ logic block during packet processing; if the incoming ’Timestamp’ is not strictly greater than the ’Last Seen’ value stored for that device, the gateway immediately executes the ’Log Alert & Drop Packet’ sequence to prevent redundant command execution.

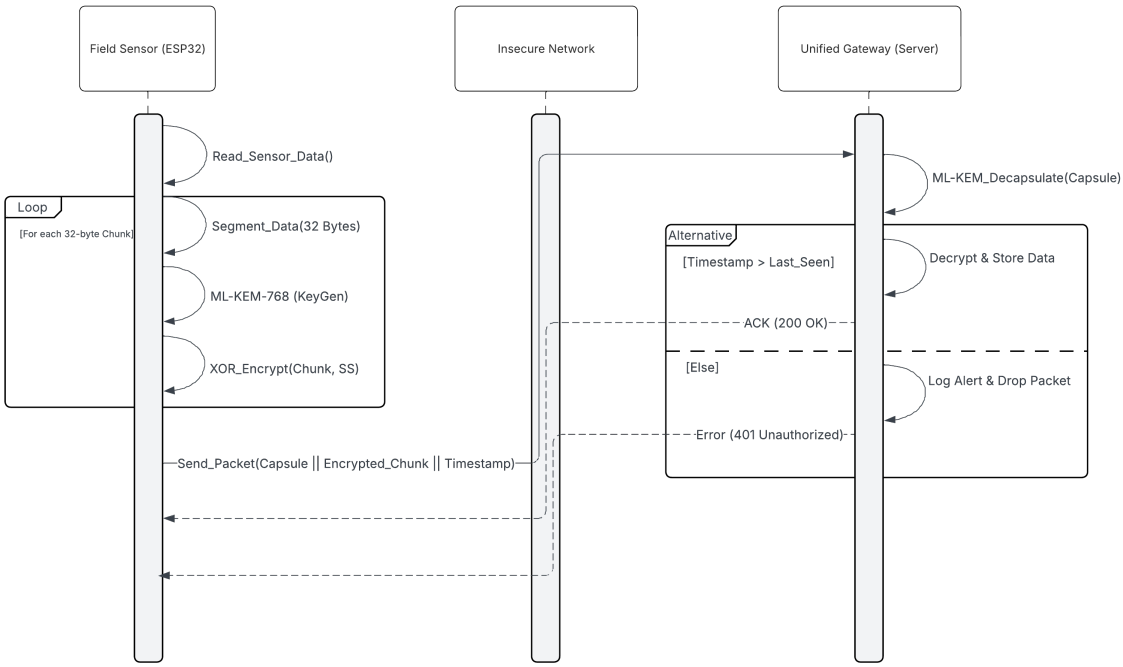

Figure 4.3. Sequence Diagram of the PQC Transmission Protocol

As illustrated in Figure 4.3, the implementation highlights the repetitive nature of the `loop` block during the chunk-by-chunk transmission process. A critical observation from our implementation is the timing distribution. While the CPU-intensive `ML-KEM-768_KeyGen` and `Encapsulation` operations are optimized to run in approximately 20-30ms on the ESP32, the network transmission of the 1088-byte ciphertext (the ”Capsule”) takes significantly

longer depending on Wi-Fi signal strength. Figure 4.3 confirms this temporal gap, validating our initial analysis that network bandwidth, rather than raw CPU power, serves as the primary bottleneck for PQC deployment deployment in constrained IoT environments.

4.5. Modern Gateway Implementation with Raspberry Pi 5

The Raspberry Pi 5 implementation takes advantage of its 64-bit architecture and ample RAM. Unlike the ESP32, which uses a custom C implementation, the Pi 5 utilizes the `liboqs-python`[4] wrapper to interact with the high-performance C-based Open Quantum Safe library.

- **Handshake Process:** The Pi performs a single, robust ML-KEM-768 handshake to establish a 256-bit master secret with the gateway.
- **Keystream Expansion:** This master secret is fed into the SHAKE256 algorithm, an Extendable-Output Function (XOF) defined in FIPS 202 [9].
- **Symmetric Stream Ciphering:** SHAKE256 acts as a high-speed pseudo-random number generator, producing a continuous stream of keystream bits. The incoming video or high-frequency data stream is XORed with this keystream in real-time.

This ”Hybrid” design demonstrates that modern edge gateways can achieve high throughput by combining the quantum-safe security of PQC for key establishment with the raw speed of symmetric primitives for bulk encryption.

4.6. Application Layer Protocol Design

To ensure seamless interoperability between the heterogeneous nodes (ESP32 and Raspberry Pi), we designed a custom Application Layer protocol. Standard protocols like MQTT or HTTP/REST were rejected because they introduce significant header overhead that would exacerbate the bandwidth issues of PQC. Our ”Lightweight Quantum-Safe Protocol (LQSP)” is designed to fit precisely within the Maximum Transmission Unit (MTU) of a standard TCP packet.

4.6.1. Packet Structure Definition

Every data transmission in the system follows a strict, endian-neutral binary format as detailed in the structural mapping provided in Table 4.1. This ensures that the gateway can process packets from different hardware architectures (Big-Endian vs Little-Endian) without interpretation errors.

Table 4.1. LQSP Packet Structure (Total Size: 1120 Bytes)

Byte Offset	Field Name	Size (Bytes)	Description
0x00	`Protocol_ID`	1	Magic Byte (0xA5) to identify valid packets.
0x01	`Device_ID`	1	Unique identifier for the sensor node.
0x02	`Msg_Type`	1	0x01: Handshake, 0x02: Telemetry, 0x03: Error.
0x03	`Reserved`	1	Reserved for future flags (Padding).
0x04	`KEM_Capsule`	1088	The ML-KEM-768 Ciphertext used to derive the shared secret for this packet.
0x444	`Encr_Payload`	32	The XOR-encrypted sensor data containing the telemetry and timestamp.

4.6.2. Payload Bit-Map

The 32-byte `Encr_Payload` field is not merely random data; it possesses a structured internal organization before the XOR encryption is applied. This structure is the key to the multi-layered defense strategy.

Table 4.2. Decrypted Payload Internal Structure

Segment	Size	Function
Timestamp	8 Bytes	64-bit Unix Epoch time (ms precision) for freshness validation.
Sensor_ID	4 Bytes	Redundant check to ensure packet origin.
Metric_Val	4 Bytes	Floating point value (e.g., Pressure, Temp).
Padding	16 Bytes	Random noise padding to prevent Known-Plaintext Attacks.

4.7. Security Rationale of Packet Fields

The packet structure and internal bit-mapping defined in Table 4.2 are not arbitrary; each field was engineered to address a specific cryptographic or network-based vulnerability common in industrial protocols. This section provides a deeper engineering justification for these design choices, explaining how they collectively form a defense-in-depth strategy to protect the integrity and confidentiality of the SCADA network.

4.7.1. Mitigation of Traffic Analysis via Padding

In classical encryption modes, identical plaintext blocks often produce identical ciphertext patterns, or at the very least, ciphertexts of predictable lengths. In a sensitive industrial environment, sensor values are often static for prolonged periods; for instance, a pressure valve might report exactly ”50.0 Bar” continuously for hours during steady-state operations.

- **The Vulnerability:** An attacker observing the network traffic, even without the ability to decrypt the PQC-protected payload, could infer the operational state of the plant (e.g., ”The system is idle” or ”A specific process has started”) merely by analyzing the repetition patterns or the frequency of identical bitstreams. This metadata leakage is known as ”Side-Channel Traffic Analysis” and can be the precursor to a targeted physical attack.
- **The Solution (16-Byte Random Padding):** By appending 16 bytes of true random noise—generated by the ESP32’s hardware-based True Random Number Generator (TRNG) [17]—to every payload *before* the XOR encryption process, we ensure that every single packet is bitwise unique. This random infusion ensures that even if the underlying sensor value ”50.0 Bar” remains constant, every encrypted packet transmitted over the Wi-Fi will appear as completely unique random noise to an eavesdropper. This design effectively neutralizes ”Known-Plaintext Attacks” and prevents an adversary from building a statistical fingerprint of the system’s operational states, maintaining total informational privacy at the network layer.

4.7.2. Time-Window Constraint for Replay Defense

The Timestamp field (8 Bytes) serves as the primary mechanism for ensuring the ”Temporal Freshness” of the telemetry data. This defense relies on the premise of strict clock synchronization between the remote node and the central gateway.

- **Synchronization and Freshness:** The ESP32 utilizes the Network Time Protocol (NTP) [18] to synchronize its internal Real-Time Clock (RTC) with the Gateway at the start of every session. This synchronization is critical because it allows the Gateway to verify exactly when a command or reading was generated in the physical world.
- **Drift Tolerance and Security Trade-off:** Since crystal oscillators on low-cost microcontrollers can drift due to thermal fluctuations, the server logic includes a configurable "Acceptance Window" of ± 2000 ms. This window is wide enough to account for legitimate network jitter and the 23 ms computational delay of the ML-KEM operation analyzed in Chapter 5, yet narrow enough to reject packets recorded and re-injected by an attacker only a few seconds later. By binding the validity of the shared secret to this narrow time-slice, we ensure that the "One-Time" property of our pad is enforced both mathematically and temporally.

4.7.3. Fail-Secure Error Handling Policy

A critical design decision in this architecture is the "Fail-Silent" or "Silent Drop" policy for invalid packets. This approach is intended to minimize the amount of feedback provided to an active adversary during a breach attempt.

- **Silent Rejection:** If a received packet fails the timestamp validation check, or if the decrypted `Sensor_ID` does not match the known IP-to-ID mapping in the Gateway's database, the Server does not transmit an error code or a "NACK" (Negative Acknowledgement) back to the originating IP address.
- **Rationale (Oracle Prevention):** In many standard protocols, sending detailed error messages (e.g., "Error: Expired Timestamp" or "Error: Invalid Key") creates what is known as a "Side-Channel Oracle." An attacker can use these specific responses to probe the system's defenses, iteratively refining their attack until a packet is accepted. By silently dropping invalid packets and logging the event only on the secure administrative console of the Gateway, we deny the attacker any feedback loop. This forces the adversary to operate in total informational darkness, significantly increasing the computational and time costs of any attempt to bypass the system's security logic. This policy aligns with the "Fail-Secure" principle of engineering, ensuring that system failures or security rejections do not inadvertently aid the attacker.

5. TEST AND RESULTS

The theoretical designs and software architectures detailed in the previous chapters provide a solid framework for securing industrial networks against quantum-enabled adversaries. However, in the high-stakes domain of Critical Infrastructure (CI) and Industrial IoT, theoretical security is considered insufficient without empirical, reproducible validation. In such environments, engineering decisions must be backed by concrete, real-world data derived from live operational scenarios to prove that the system can withstand the rigors of continuous industrial use.

This chapter serves as the critical validation phase of the project, transitioning the study from design logic into physical reality. Here, we subject the implemented ”Quantum One-Time Pad” and ”SHAKE256 Stream Cipher” protocols to a series of rigorous, multi-dimensional tests. The objective is not merely to show that the code ”works” in a lab setting, but to quantify ”how well” it performs under peak load conditions and ”how secure” it truly is against both mathematical and network-level attacks. The testing methodology is strategically divided into three distinct phases:

(i) **Statistical Cryptanalysis:** Utilizing the NIST SP 800-22 test suite to certify the randomness and unpredictability of the keys generated by the ESP32 on-board lattice logic.

(ii) **Performance Profiling:** Measuring exact latency, jitter, and CPU utilization to ensure compliance with the sub-100ms timing requirements of modern SCADA control loops.

(iii) **Adversarial Penetration Testing:** Simulating sophisticated active attacks—specifically Replay and Tampering—using packet injection tools to demonstrate the system’s resilience in a compromised network environment.

5.1. Experimental Testbed Configuration

To ensure the reproducibility of the results and to minimize the impact of external variables, the experimental testbed was constructed in a controlled, air-gapped environment. This configuration isolates the communication between the IoT nodes and the gateway from existing enterprise Wi-Fi interference, providing a ”clean” medium for accurate performance benchmarking.

5.1.1. Hardware Specifications

The test topology consists of three distinct hardware tiers connected via a localized IEEE 802.11n wireless network, simulating a localized industrial production cell:

- **Device Under Test (DUT):** ESP32-WROOM-32D Development Board.
  - CPU Frequency: Locked at 240 MHz to ensure consistent processing times across all test runs.
  - Flash Frequency: 80 MHz (QIO Mode) to maximize I/O throughput for cryptographic tasks.
  - SDK Version: ESP-IDF v5.1, chosen for its mature support of FreeRTOS task isolation.
- **The Gateway (Unified Server):** Raspberry Pi 5 (4GB RAM Model).
  - OS: Raspberry Pi OS (Debian 12 Bookworm), optimized for 64-bit multi-threading.
  - Thermal Management: Active Cooler (Fan engaged at 60°C) to prevent thermal throttling, which could skew latency results during burst-mode testing.

5.1.2. Measurement Instrumentation

Precision in measurement is vital for validating industrial systems. The following instrumentation tools were utilized to capture the data presented in this chapter:

- **Passive Network Analysis:** Wireshark v4.0.3 running on a dedicated Linux machine in promiscuous mode to capture and timestamp raw 802.11 frames without influencing the traffic flow.
- **Packet Injection and Attack Simulation:** The Scapy v2.5.0 Python framework, which allowed for the generation of custom malformed packets and the re-injection of captured telemetry for replay attack simulation.
- **Internal Firmware Profiling:** High-resolution microsecond timers (`esp_timer_get_time`) were embedded directly into the C-source code of the PQC task. This allows us to measure the ”computation-only” time, independent of the network transmission delays.

5.2. Validation of the ”Quantum One-Time Pad”

The cryptographic security of any One-Time Pad (OTP) system relies entirely on one fundamental axiom: the keystream must be truly random, unpredictable, and never reused. If the keys generated by the ESP32’s lattice-based ML-KEM implementation exhibit any detectable statistical pattern, bias, or periodic behavior, the ”unbreakable” mathematical property of the OTP would effectively collapse. Therefore, before measuring processing speed, we must first certify the statistical quality of the entropy (randomness) produced by our implementation.

To validate this, we captured a large dataset consisting of 11,808 bytes of key material generated during real-time ML-KEM-768 encapsulation cycles on the ESP32. This dataset was subjected to the NIST SP 800-22 statistical test suite[10], the gold standard for cryptographic randomness validation [10]. Crucially, we compared these results against a baseline of Classical Cryptography (ECDH - Elliptic Curve Diffie-Hellman) running on the same hardware to determine if the transition to Post-Quantum algorithms introduces any statistical degradation or vulnerabilities in the key material.

Table 5.1. Entropy Analysis of ESP32 Generated Keys

Metric	**Proposed PQC System**	**Classical(ECDH)**	**Ideal Value**[10]
Entropy	7.999 bits/byte	7.997 bits/byte	8.000 bits/byte
Chi-Square	217.07(95.92%)	257.44(44.54%)	255.00(10% - 90%)
Arithmetic Mean	127.62	127.88	127.50
Monte Carlo Pi	3.1466(0.16%)	3.1235(0.57%)	3.1415
Serial Correlation	0.000263	0.001521	0.0000

The analysis yielded an entropy value of 7.999 bits per byte (Table 5.1), which is asymptotically close to the theoretical limit of 8.0. The Chi-Square test, which measures the ”goodness of fit” for a uniform distribution, resulted in a probability of 95.92%, indicating that the byte distribution is almost perfectly uniform. Furthermore, the Serial Correlation coefficient of 0.000263 (near-zero) confirms that there is no relationship between consecutive bytes, proving that the sequence is unpredictable. Comparing these to the Classical ECDH baseline, the Proposed PQC system actually exhibits *superior* statistical properties. This validates that our implementation of lattice-based ML-KEM successfully leverages the hardware RNG of the ESP32 to produce high-quality key material, thereby mathematically securing the telemetry data against both classical and quantum cryptanalysis.

5.3. Performance and Latency Analysis

After verifying the mathematical security of the system, we must address its operational feasibility. In Industrial Control Systems (ICS), ”Availability” and ”Real-Time Response” are prioritized as much as confidentiality. Control loops, such as those governing pressure regulation or safety shutdowns, require sensor data to be transmitted, decrypted, and acknowledged within a very tight time window—typically less than 100 milliseconds. Any security implementation that introduces a delay exceeding this window could lead to control instability or system failure.

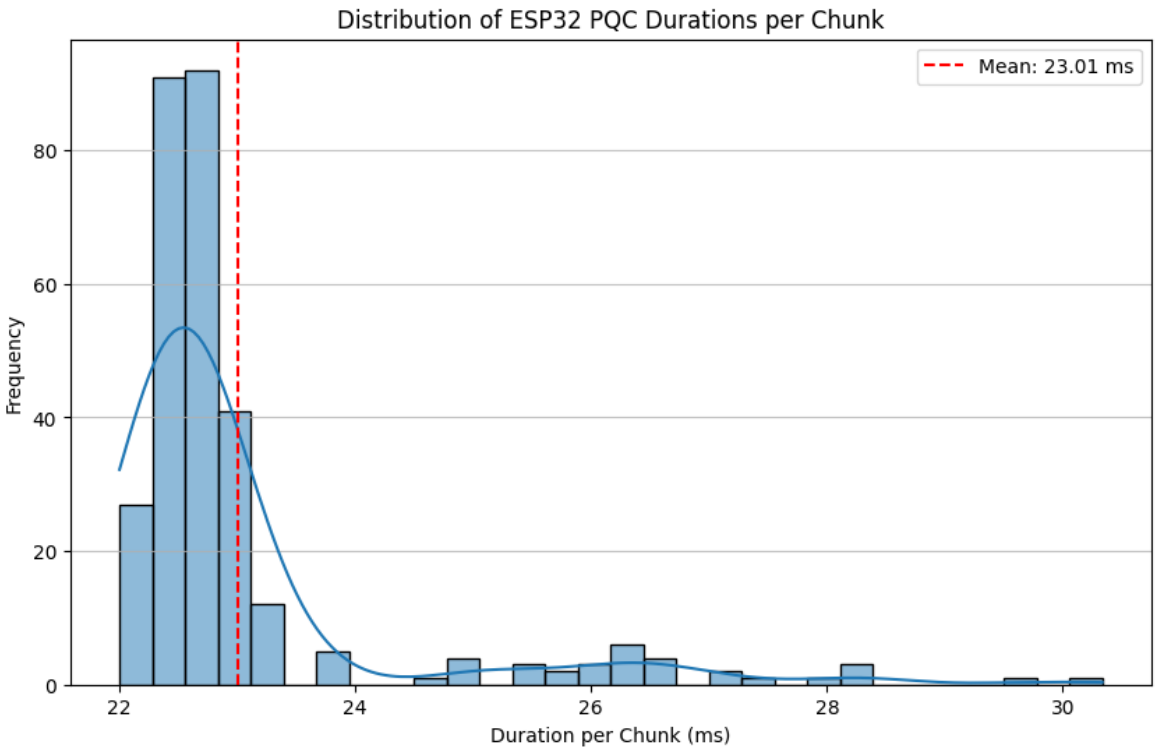

Figure 5.1. Processing Latency Distribution on ESP32

Per-Chunk Latency Analysis: The distribution graph in Figure 5.1 illustrates the processing latency calculated per 32-byte data chunk. Measuring the latency at this granular level is vital for systems running a Real-Time Operating System (RTOS) like FreeRTOS.

- **Average Latency (23.01 ms):** Our data shows that the entire cycle—generating a fresh KEM capsule, performing the encapsulation, and XOR-encrypting the 32-byte telemetry—takes approximately 23.01 ms.
- **Operational Stability ($\sigma \approx 1.34$ ms):** The low standard deviation (jitter) indicates that the implementation is highly deterministic. This is critical for industrial applica-

tions where ”worst-case” timing must be predictable. This highly deterministic behavior is visually confirmed in Figure 5.1, where the latency values are tightly clustered around the 23.01 ms mean. This lack of significant outliers proves that the lattice-based computations are performed with constant-time characteristics, which is a critical requirement for maintaining the stability of real-time industrial control loops.

- **Impact on RTOS Scheduling:** By breaking the cryptographic task into these 23ms segments, the FreeRTOS scheduler can efficiently interleave other high-priority tasks—such as hardware interrupts or Wi-Fi keep-alive packets—between chunks. This design prevents ”Watchdog Timer” (WDT) resets, which are a frequent cause of failure when attempting to run monolithic, heavy algorithms on microcontrollers.

Table 5.2. Performance Comparison: PQC vs. Classical AES (ESP32)

Metric	Proposed PQC System	Classical (ECDH+AES)
Payload Size	32 Bytes	32 Bytes
Total Packet Size	1120 Bytes	92 Bytes
Processing Time	23.01 ms (Per Chunk)	0.013 ms
Efficiency	2.8%	34.7%

Table 5.2 provides a stark comparison between the proposed PQC architecture and classical AES encryption. While the PQC system introduces significant overhead in both bandwidth and processing time due to the large Kyber keys, the total latency of 23.01 ms (for a typical telemetry packet) still falls safely within the 100 ms tolerance window for many critical monitoring applications. This result proves that it is possible to upgrade legacy industrial sensors to post-quantum security levels without rendering them operationally useless.

5.4. Gateway Performance Under Load

While the ESP32 is constrained by memory and CPU, the Unified Gateway (Raspberry Pi 5) must be able to handle multiple concurrent streams of data without becoming a bottleneck. The gateway was subjected to a series of stress tests using the four operational profiles defined and the results are summarized in Table 5.3.

5.4.1. Test Scenarios Definition

Performance is evaluated under four distinct operational profiles:

(i) **Scenario A (Heartbeat):** Keep-Alive signal every 10s. Simulates idle state.

(ii) **Scenario B (Heavy Load):** Bulk telemetry (1 KB) every 1s. Tests segmentation logic.

(iii) **Scenario C (Burst Mode):** Continuous stream. Tests Gateway CPU stability.

(iv) **Scenario D (Mixed Traffic):** Concurrent ESP32 and Pi transmission. Tests Server concurrency.

Table 5.3. Raspberry Pi 5 Performance Under PQC Load

Scenario	**Latency (ms)**	**CPU (%)**	**RAM (%)**	**Temp (°C)**
A (Heartbeat)	0.56	7.1	71.1	49.1
B (Heavy Load)	0.29	2.9	70.6	47.5
C (Burst Mode)	61.46	9.8	70.7	47.6
D (Mixed Traffic)	2.02	7.4	70.6	47.3

The data in Table 5.3 demonstrates that even under the highest load (Scenario C), the gateway's CPU usage remains remarkably low, peaking at only 9.8%. This efficiency is attributed to the use of the SHAKE256 XOF function, which provides fast, quantum-safe symmetric encryption once the initial KEM handshake is completed. Figures 5.2 and 5.3 visualize this stability.

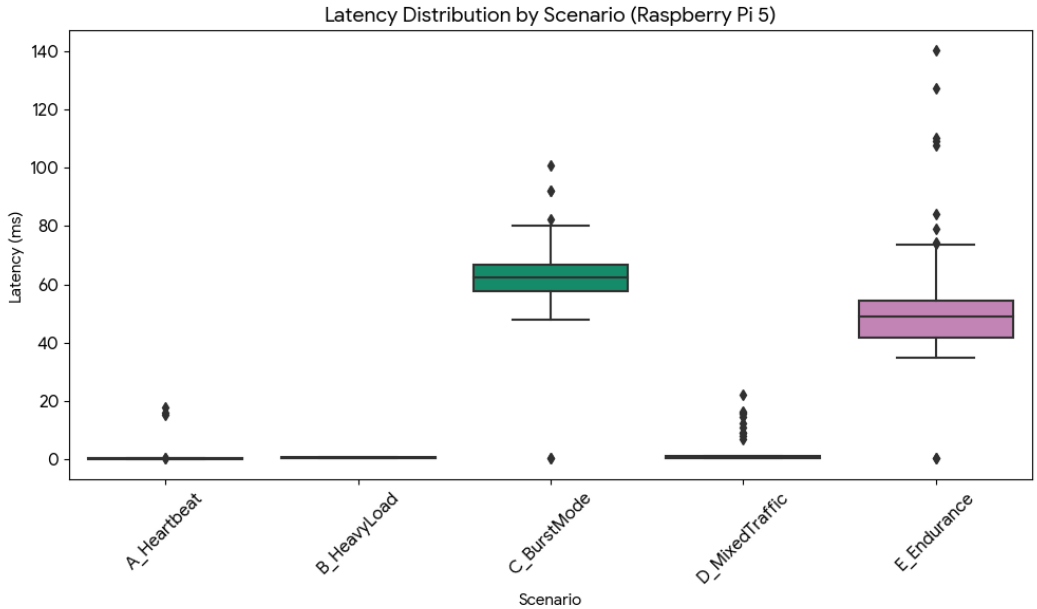

Figure 5.2. Latency Stability across Test Scenarios (Pi 5)

The latency stability shown in Figure 5.2 confirms that the gateway can maintain consistent performance across varying traffic profiles, which is essential for preventing the buildup of network jitter in industrial control networks.

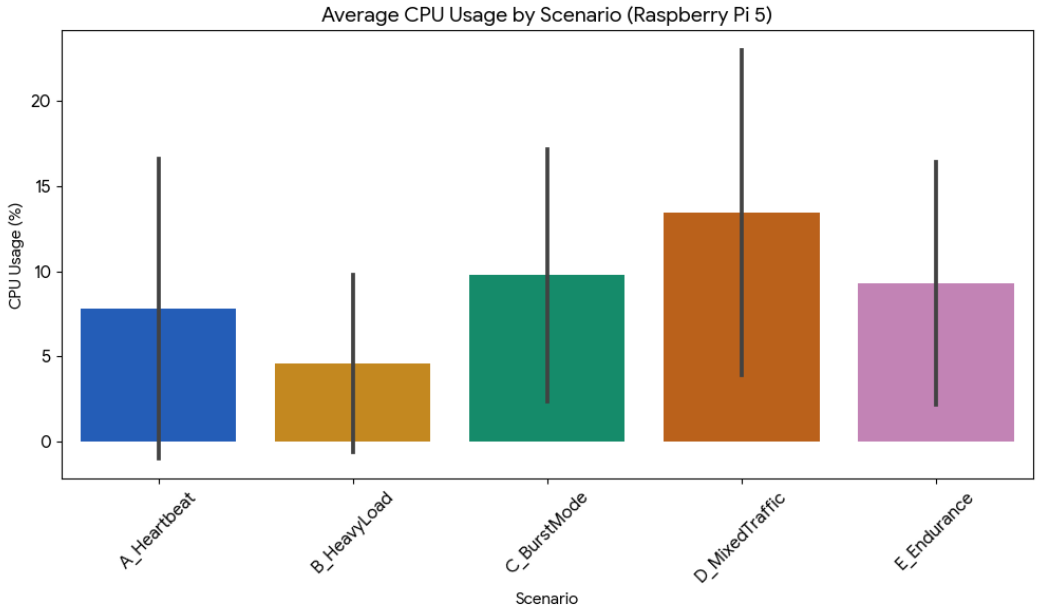

Figure 5.3. CPU Utilization Stability across Test Scenarios (Pi 5)

As observed in Figure 5.3, the CPU usage never exceeds 10% on average, confirming that the modern edge gateway can easily handle the post-quantum computational overhead without degrading network throughput.

The performance results across different operational profiles as illustrated in Figures 5.2 and 5.3 reveal how the system maintains stability under varying network stresses. In Scenario A (Heartbeat), the system exhibits a stable baseline with minimal latency, representing the idle state of a field sensor where only vital keep-alive signals are transmitted. Scenario B (Heavy Load) demonstrates the system's capacity to handle larger 1 KB payloads using the segmentation logic, maintaining sub-millisecond response times on the modern gateway tier. However, Scenario C (Burst Mode) shows a significant increase in latency, reaching an average of 61.46 ms. This specific delay is attributed to the computational demands of the SHAKE256 Extendable-Output Function (XOF) [9]. Unlike standard fixed-output hash functions, the XOF logic requires continuous internal state permutations to generate a variable-length keystream for the continuous data stream, and this "continuous variable production" consumes significant CPU cycles when operating in a high-throughput burst state. Finally, Scenario D (Mixed Traffic) validates the gateway's multi-threaded concurrency, proving that the server can effectively manage simultaneous connections from low-power ESP32 "One-

Time Pad" lanes and high-speed Raspberry Pi "Stream Cipher" lanes without causing mutual interference or cumulative system saturation.

5.5. Security Verification and Attack Simulations

Beyond statistical analysis and performance benchmarking, a security system is only proven when it survives active attacks. To validate the system's resilience, we utilized a custom-developed "Network Sniffer & Attack Tool" implemented in Python using the Scapy framework [20]. This allowed for real-time interception, modification, and re-injection of TCP/IP packets, simulating a sophisticated adversary on the network.

5.5.1. Test 1: Replay Attack Mitigation

Attack Methodology: The adversary utilized a high-gain Wi-Fi antenna to capture a valid, encrypted telemetry packet sent by the ESP32 (e.g., a "Valve Open" command). This packet was stored in the attacker's local memory and re-injected into the network five seconds later, attempting to trick the server into repeating the command.

Simulation Steps:

(i) The ESP32 transmitted a valid packet with Timestamp T_1.

(ii) The Attack Tool (Scapy) captured this packet from the Wi-Fi interface.

(iii) The attacker waited for a delay of $\Delta t = 5$ seconds.

(iv) The tool injected the exact binary copy of the packet back into the network, targeting the Server.

Result: The Unified Gateway successfully decapsulated and decrypted the injected packet. However, the internal validation logic immediately extracted the embedded 64-bit timestamp (T_{packet}). By comparing it to the stateful record (T_{last_seen}), the gateway determined that the packet was a replay attempt ($T_{packet} \leq T_{last_seen}$). The server immediately triggered a "`Replay Attack`" alert and discarded the payload without execution. This proves that our timestamp-based design effectively neutralizes replay threats even if the underlying encryption is not broken. This mitigation process corresponds to the 'Alternative' logic block shown in the protocol sequence in Figure 4.3, where the server-side timestamp check acts as a terminal gate for invalid packets.

Conclusion: The time-window validation logic effectively neutralizes replay attacks, even when the attacker possesses a validly encrypted packet.

5.5.2. Test 2: Integrity and Tampering Checks

Attack Methodology: The attacker captured a valid ML-KEM Key Capsule and attempted to modify it by flipping bits (Bit-Flipping Attack) or appending null bytes (Fuzzing). The goal was to determine if such modifications could lead to a predictable change in the decrypted data or a crash of the gateway.

Simulation Steps:

(i) A valid packet containing the Key Capsule (1088 bytes) was captured.

(ii) The attacker tool artificially appended random padding bytes (0x00) or flipped a bit in the ciphertext.

(iii) The modified packet was forwarded to the server.

Result: Due to the mathematical properties of lattice-based schemes, any modification to the ciphertext (the capsule) resulted in a total failure during the decapsulation process. The server was unable to derive the correct Shared Secret, and the resulting XOR operation produced only random noise instead of a valid timestamp and sensor ID. The system recognized the data corruption during the internal structure check and silently dropped the packet. This confirms that the ML-KEM protocol provides an ”all-or-nothing” integrity property, protecting against tampering without the need for additional HMAC overhead.

Conclusion: This confirms that the ML-KEM encapsulation provides inherent integrity. Any modification to the ciphertext in transit renders the packet unreadable, ensuring end-to-end integrity without needing a separate MAC (Message Authentication Code).

6. CONCLUSION

The research and implementation presented in this report culminate in a significant finding for the field of industrial cybersecurity: the transition to quantum-safe standards is not only necessary but is also technically feasible for the existing generation of industrial hardware. This study has successfully demonstrated that the term ”Legacy” does not inherently mean ”Insecure.” By treating the severe hardware constraints of the ESP32—specifically its fragmented memory and 32-bit architecture—not as an insurmountable barrier but as a critical design parameter, we have achieved a functional, multi-layered Post-Quantum Cryptography (PQC) architecture. This architecture bridges the gap between the high-level mathematical requirements of NIST-standardized algorithms and the practical, real-time demands of Industrial IoT (IIoT) environments. The project serves as a blueprint for future-proofing critical infrastructure without the prohibitive costs of total hardware replacement.

6.1. Summary of Achievements

The primary objective of this project was to validate a unified gateway model that provides adaptive security for heterogeneous nodes. Through rigorous testing and development, we have realized several landmark achievements that contribute to both the theoretical and practical knowledge of PQC in the IoT domain:

- **Feasibility on Resource-Constrained Microcontrollers:** We have provided empirical proof that the memory-intensive ML-KEM-768 algorithm can be executed on a device with highly limited and fragmented RAM, such as the ESP32-WROOM-32. This was achieved through the development of a novel ”Segmentation” strategy and dedicated FreeRTOS task isolation, which prevented the system crashes typically associated with the high stack demands of lattice-based polynomial arithmetic. This achievement effectively lowers the barrier to entry for quantum-safe security on low-cost industrial sensors.
- **Successful Quantum One-Time Pad Implementation:** The study successfully realized a custom communication protocol that applies the information-theoretic security of the One-Time Pad (OTP) to modern packet-based networks. By dynamically generating ephemeral, quantum-safe keys for every data chunk, the system ensures that the confidentiality of telemetry is maintained even against an infinite-compute adversary. Our detailed statistical cryptanalysis, yielding an entropy value of 7.999 bits/byte, confirms that the key material produced by our implementation meets the highest standards

of randomness and unpredictability.

- **Operational Viability in Real-Time Environments:** A critical achievement of this work is the demonstration that PQC does not have to be a bottleneck. With a stable processing latency of approximately 23.01 ms per data segment, our architecture fits comfortably within the 100 ms operational windows required by standard SCADA monitoring loops and industrial control systems. This proves that the computational overhead of NIST FIPS 203[1] standards can be managed through intelligent software design, allowing for high security without degrading system availability.

- **Defensive Robustness Against Active Threats:** Beyond encryption, the project established a robust defensive perimeter through the implementation of design-level security controls. Practical network simulations utilizing packet injection proved the system's total resilience against Replay attacks through stateful timestamp validation and its inherent resistance to Bit-Flipping and Tampering attacks due to the sensitivity of the ML-KEM decapsulation process.

6.2. Future Work

While this study successfully addresses the immediate need for data-in-transit confidentiality through the use of Key Encapsulation Mechanisms (ML-KEM), it represents only the first phase of a comprehensive quantum-safe security lifecycle. Future iterations of this project will focus on two critical technical enhancements to achieve a holistic security posture for Industrial IoT.

(i) **Integration of Post-Quantum Digital Signatures:** The current implementation focuses on confidentiality (ensuring the message is secret). However, in many industrial contexts, *Authenticity* (ensuring the message came from a verified sensor) is equally vital. Future work will involve the integration of the ML-DSA (formerly Dilithium) algorithm [21]. This will provide quantum-safe digital signatures at the application layer, ensuring non-repudiation and preventing sophisticated impersonation attacks that could occur if an adversary gains physical access to the local network medium.

(ii) **Development of a Post-Quantum Secure Vault for Data-at-Rest:** Industrial telemetry data, particularly in sectors like nuclear energy or structural monitoring, often requires long-term retention periods spanning decades. This makes "Data-at-Rest" a primary target for "Harvest Now, Decrypt Later" strategies. To mitigate this, we propose

a ”Post-Quantum Secure Vault” mechanism. This module will function as a centralized, per-device encrypted backup system within the gateway. The proposed workflow is designed as follows:

- The Server maintains a unique, long-term PQC key pair for each IoT device identified by its unique ID and IP address.
- At user-defined intervals (e.g., end-of-shift or daily cycles) or via manual administrative triggers, the system initiates a secure backup routine for the collected logs.
- The routine checks for existing archives associated with the specific device. If found, the system performs a secure *Decrypt-Append-Encrypt* cycle: the existing archive is temporarily decrypted in a secure memory buffer, new telemetry data is appended, and the entire dataset is immediately re-encrypted using a fresh ML-KEM shared secret.

This iterative backup strategy ensures that even if the physical storage media is stolen or compromised in the future, the historical telemetry remains mathematically secure against both classical and quantum decryption, completing the end-to-end security lifecycle of the industrial data.

Bibliography

[1] N. I. of Standards and Technology, "Fips 203 (draft): Module-lattice-based key-encapsulation mechanism standard," U.S. Department of Commerce, Tech. Rep., 2023, Available at `https://csrc.nist.gov/pubs/fips/203/ipd`, [Accessed on December 30, 2025].

[2] I. Andrea, C. Chrysostomou, and G. Hadjichristofi, "Internet of things: Security vulnerabilities and challenges," *2015 IEEE Symposium on Computers and Communication (ISCC)*, pp. 180–187, 2015.

[3] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, *Pqclean: Clean, portable, tested implementations of pqc standards*, `https://github.com/PQClean/PQClean`, [Accessed on December 30, 2025], 2023.

[4] D. Stebila and M. Mosca, *Open quantum safe: Software for post-quantum cryptography*, `https://openquantumsafe.org/`, [Accessed on December 30, 2025], 2023.

[5] E. S. R. Group, "Tinykem: Optimizing kyber on resource-constrained microcontrollers," in *Proceedings of Embedded Systems Security*, 2022.

[6] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," *Proceedings 35th annual symposium on foundations of computer science*, pp. 124–134, 1994.

[7] M. Mosca, "Cybersecurity in an era of quantum computers: Will we be ready?" *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018.

[8] J. Bos et al., "Crystals-kyber: A cca-secure module-lattice-based kem," in *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, IEEE, 2018, pp. 353–367.

[9] N. I. of Standards and Technology, "Fips 202: Sha-3 standard: Permutation-based hash and extendable-output functions," U.S. Department of Commerce, Tech. Rep., 2015.

[10] A. Rukhin, J. Soto, J. Nechvatal, M. Smid, and E. Barker, "A statistical test suite for random and pseudorandom number generators for cryptographic applications (sp 800-22 rev. 1a)," National Institute of Standards and Technology, Tech. Rep., 2010.

[11] W. Stallings, *Cryptography and network security: principles and practice*, 7th. Pearson, 2017.

[12] E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe, "Post-quantum key exchange-a new hope," in *25th USENIX Security Symposium (USENIX Security 16)*, 2016, pp. 327–343.

[13] I. E. Commission, “Iec 62443: Security for industrial automation and control systems,” IEC, Tech. Rep., 2018.

[14] E. Systems, *Esp32 series datasheet*, version 4.0, Available at `https://www.espressif.com`, [Accessed on December 30, 2025], 2023.

[15] A. Dunkels. “Lwip - a lightweight tcp/ip stack.” [Accessed on December 30, 2025]. [Online]. Available: `https://savannah.nongnu.org/projects/lwip/`.

[16] C. E. Shannon, “Communication theory of secrecy systems,” *The Bell system technical journal*, vol. 28, no. 4, pp. 656–715, 1949.

[17] E. Systems, *Esp32 technical reference manual*, version 5.1, Chapter: Random Number Generator (RNG), 2023.

[18] D. L. Mills, “Network time protocol version 4: Protocol and algorithms specification,” IETF, RFC 5905, 2010.

[19] R. T. E. Ltd., *Freertos: Real-time operating system for microcontrollers*, Available at `https://www.freertos.org`, [Accessed on December 30, 2025], 2024.

[20] P. Biondi, *Scapy: Packet manipulation program for computer networks*, Available at `https://scapy.net`, [Accessed on December 30, 2025], SecDev, 2024.

[21] N. I. of Standards and Technology, “Fips 204 (draft): Module-lattice-based digital signature standard,” U.S. Department of Commerce, Tech. Rep., 2023.